De una neurona a tu primera red

Neural Networks: Deep Learning

UIDE · Maestría en Ciencia de Datos

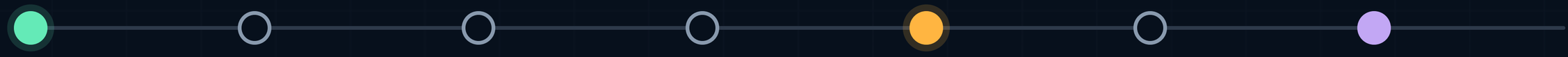
Semana 1



La semana en dos sesiones

Primero la idea, después la demo y al final la práctica en Colab. Las prácticas son en clase y no tienen nota.

Viernes 23 18:00–20:00



18:00	18:10	18:25	18:48	19:02	19:22	19:50
Pre-quiz	01 · Aprender de ejemplos	02 · La neurona	03 · Cómo aprende	Práctica NB01	04 · Error y gradiente	Cierre y lectura

Sábado 24 08:00–12:00



08:00	08:15	08:55	09:40	09:55	10:30	10:50	11:30	11:45
Repaso	05 · Redes	06 · Retro-propagación	Pausa	07 · Entrenar bien	08 · Keras	Práctica NB02	09 · Métricas	Quiz final

práctica en Colab · quiz tipo Kahoot · lectura de tendencia

Antes de empezar: ¿de dónde partimos?

8 preguntas cortas tipo Kahoot. **No es nota**: es tu punto de partida para medir cuánto avanzas.

1. Escanea el código o abre **deeplearningweek1.vercel.app/?pre**
2. Escribe tu nombre y responde rápido: **más rapidez, más puntos**
3. Al final, copia tu código y **pégalo en el chat de Teams**

✓ Si no sabes una, elige la que te parezca. Equivocarse aquí es parte del plan.



deeplearningweek1.vercel.app/?pre

El sábado vas a poder explicar estas 5 ideas

Las medimos dos veces: en el pre-quiz de hoy y en el quiz final del sábado.

1. Qué calcula una neurona
2. Para qué sirve la activación
3. Por qué una sola neurona no puede con XOR
4. Hacia dónde camina el descenso de gradiente
5. Por qué la exactitud puede engañar

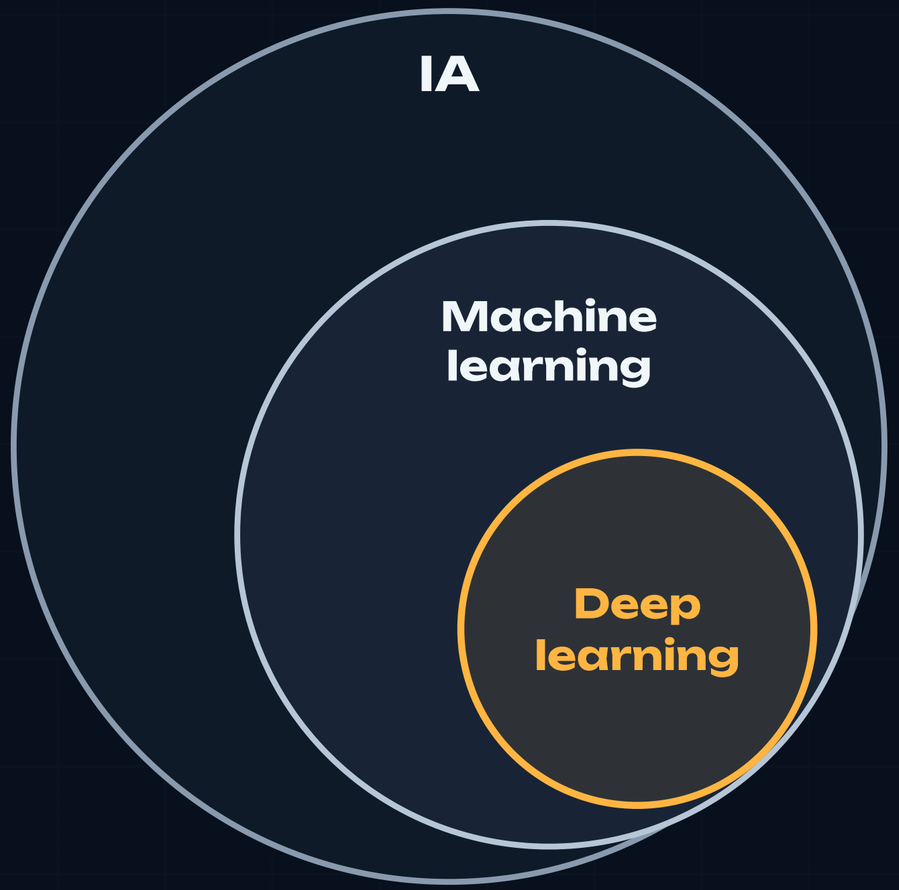
01

IA, machine learning y deep learning

Qué significa que una máquina aprenda de ejemplos.

Tres círculos · Reglas o ejemplos · Dónde ya lo usas · ¿Por qué ahora?

Viernes · 15 minutos



Tres círculos, uno dentro de otro

Inteligencia artificial

Máquinas que hacen tareas que parecen inteligentes, aprendan o no.

El GPS que calcula la ruta · el rival de un videojuego

Machine learning

Aprende las reglas a partir de ejemplos.

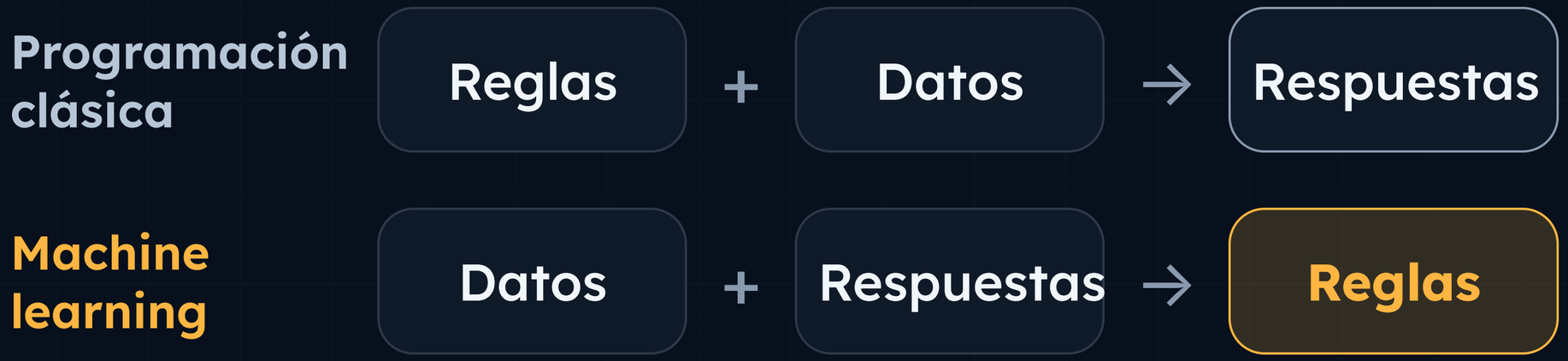
El filtro de spam · las recomendaciones de Netflix

Deep learning

Machine learning con redes de muchas capas.

Desbloquear el celular con tu cara · los subtítulos automáticos · ChatGPT

Programar reglas o aprender de ejemplos



Regla escrita: «si dice GRATIS, es spam». Y el correo de tu tienda favorita, «¡Envío gratis!», termina en spam.

Aprendida: le muestras 10 000 correos ya marcados y el modelo descubre solo qué combinación de señales delata al spam.

Intenta escribir las reglas de un gato

Para que una computadora reconozca gatos en fotos, ¿qué reglas le darías?

Si **tiene orejas puntiagudas**, es un gato — × un zorro también

Si **tiene bigotes**, es un gato — × una foca también

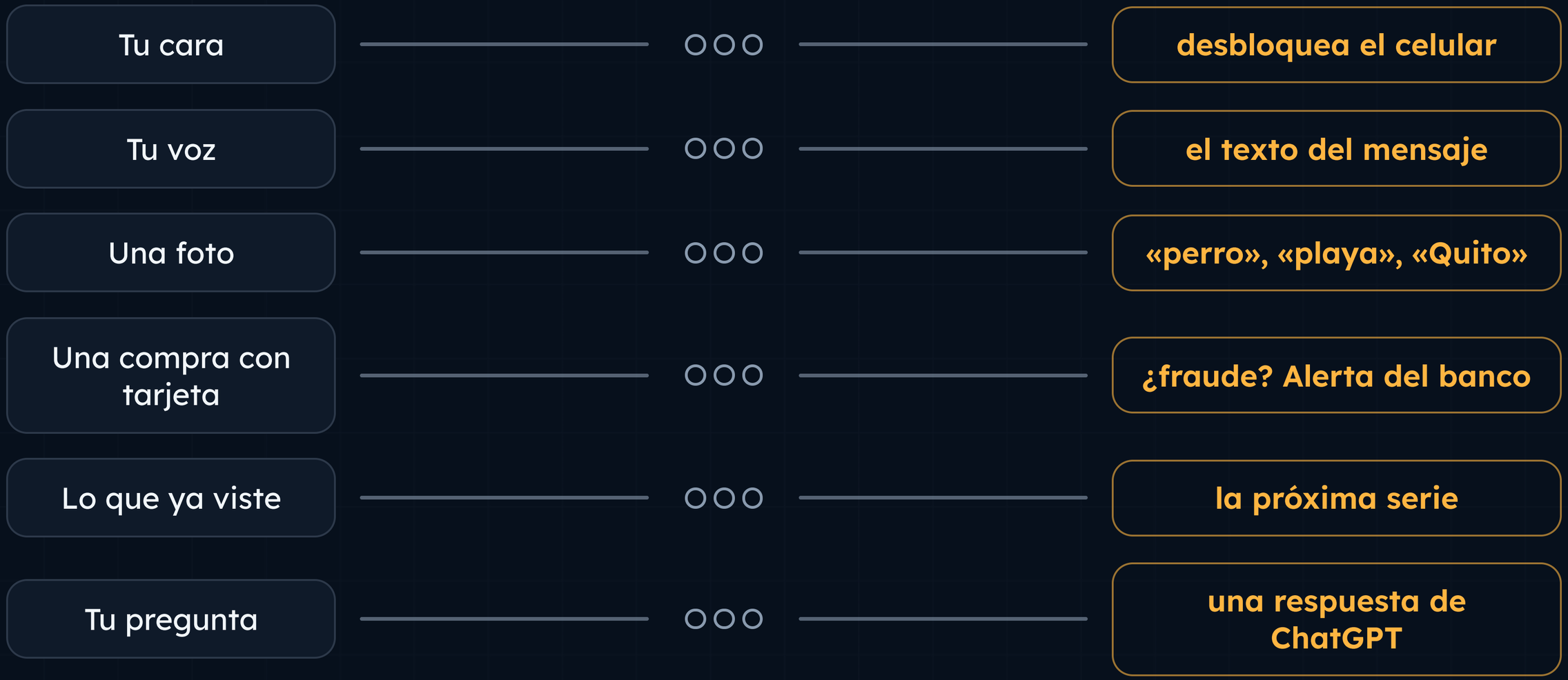
Si **es pequeño y peludo**, es un gato — × un cuy también

Si **maúlla**, es un gato — × en una foto no se oye nada

Mejor: muéstrale miles de fotos con su etiqueta, «gato» o «no gato», y deja que encuentre las señales. Eso es deep learning.

Deep learning que ya usas hoy

Todos hacen lo mismo: convierten una **entrada** en una **salida**, después de aprender de millones de ejemplos.



Las ideas tienen 60 años. ¿Por qué despegaron ahora?

1958 — El perceptrón de Rosenblatt: la primera neurona artificial que aprende

1969 — Minsky y Papert muestran su límite (XOR) y llega el primer «invierno»

1986 — Se populariza la retropropagación: redes de varias capas

2012 — AlexNet gana ImageNet entrenada con tarjetas gráficas (GPU)

2017 — El Transformer, base de los modelos de lenguaje

2022 — ChatGPT llega al público

Tres ingredientes se juntaron: **muchos datos** · **cómputo barato (GPU)** · **mejores algoritmos**

Lectura de tendencia

El cómputo dedicado a la IA crece **más de 3 veces por año** (AI Index 2026). Entonces, ¿todavía importan los mejores algoritmos? Es la pregunta de la lectura de esta semana.

02

La neurona artificial

Una calculadora con opinión: recibe datos, los pesa y decide.

- Tu nota es una neurona
- Un comité de crédito
- La cuenta a mano
- Una línea
- La activación

Viernes · 23 minutos

Tu nota final del curso ya es una neurona

Idea clave 1

Cada nota por lo que pesa, todo sumado y comparado con la exigencia.

$$\underbrace{z}_{\text{¿pasa o no?}} = \underbrace{\sum \text{nota} \cdot \text{peso}}_{\text{cada nota por lo que pesa, sumadas}} - \underbrace{70}_{\text{la exigencia}}$$

Léelo así: las notas son los **datos**, los porcentajes del sílabo son los **pesos** y la exigencia es el **sesgo** (con signo menos). Si z ≥ 0, apruebas.

Aquí los pesos los fija el sílabo. En una red, **los pesos se aprenden de ejemplos**: eso veremos hoy.



Ana: Nota final **70.0** − 70 = **z = 0.0** →

Aprueba, justo

Ana | Beto | ¿Cuánto necesita Ana en el final?

La misma cuenta decide un crédito

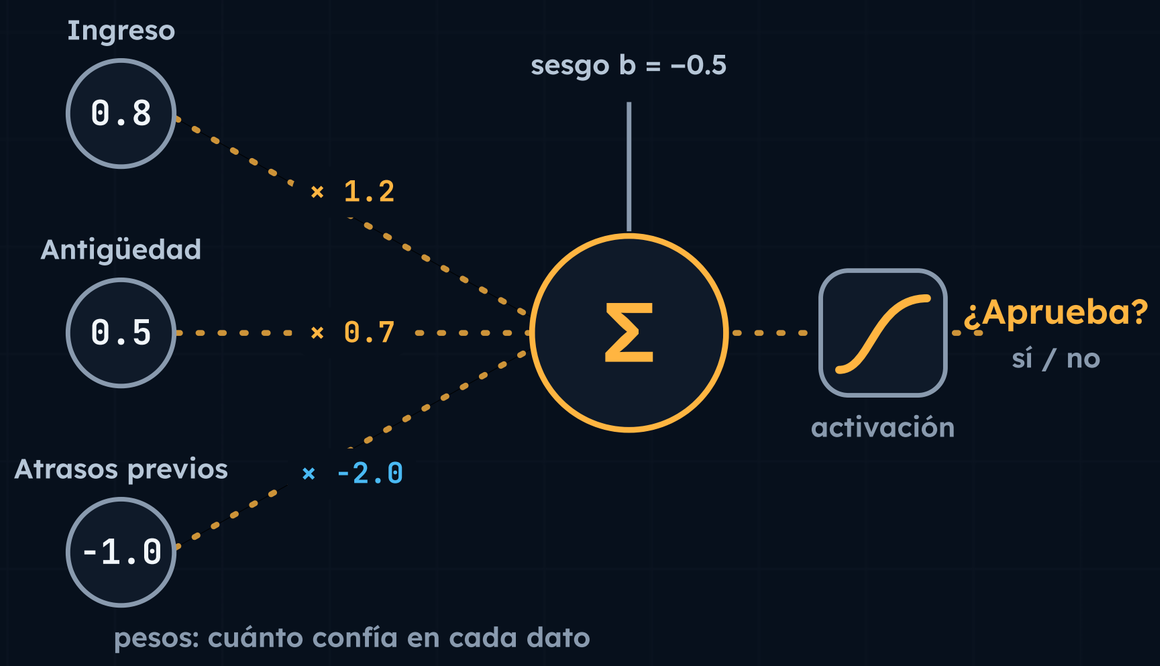
Idea clave 1

El comité de una cooperativa pesa tres datos de cada socio. Novedad: **un peso puede ser negativo** y entonces resta.

$$z = w \cdot x + b$$

puntaje · pesos · datos del socio · la exigencia

Léelo así: multiplica cada dato por su peso, suma todo y añade el sesgo.

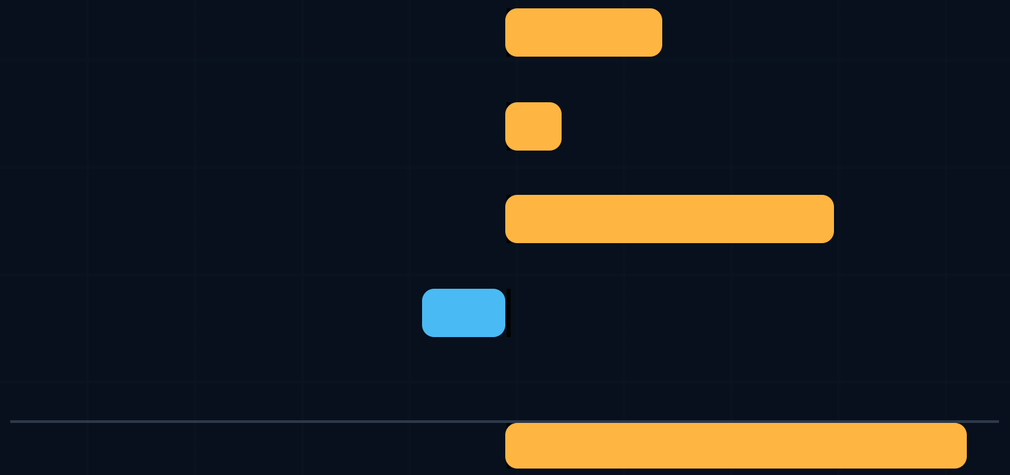


Hagamos la cuenta con el socio A

Idea clave 1

Cada dato por su peso, y después se suma todo.

Ingreso	0.8 × 1.2	+0.96
Antigüedad	0.5 × 0.7	+0.35
Atrasos previos	-1.0 × -2.0	+2.00
Sesgo	b	-0.50
Puntaje z	suma	2.81



z es positivo: **el comité aprueba**. Los pocos atrasos (–1.0 por un peso de –2.0) aportaron más que el ingreso.

Tu turno · NB01 §1 El **socio B**: ingreso 0.2, antigüedad –0.5 y atrasos 1.5. ¿Lo aprueba?

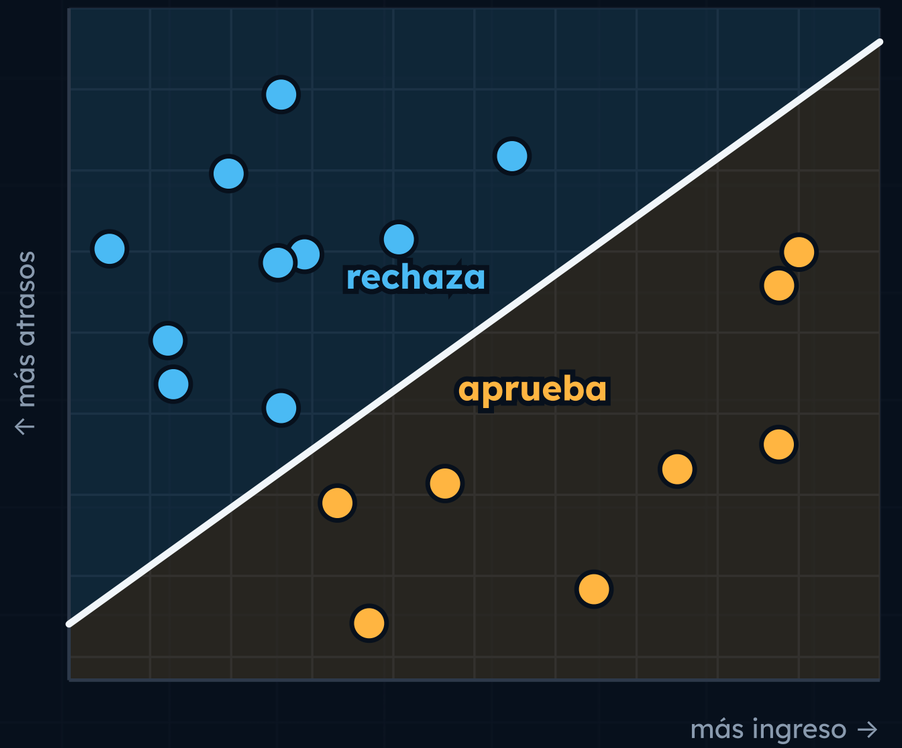
Una neurona dibuja una línea

Cada socio es un punto. La neurona parte el plano en dos con una línea recta: de un lado aprueba, del otro rechaza.

Los **pesos** giran la línea.
El **sesgo** la mueve sin girarla.
Entrenar es encontrar dónde ponerla.

● sí pagó ● no pagó

○ mal clasificado



w_1 ingreso 1.30

w_2 atrasos -1.50

b sesgo 0.05

Aciertos: 18 de 18

¡todos!

Buscar la mejor línea

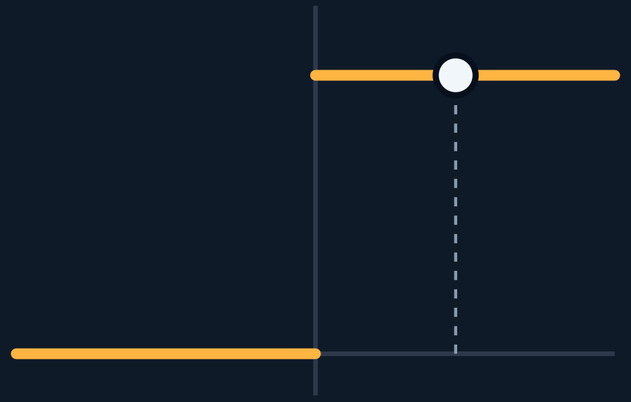
Reiniciar

La activación decide qué sale de la neurona

Idea clave 2

El mismo puntaje z pasa por tres funciones distintas. Mueve z y compara.

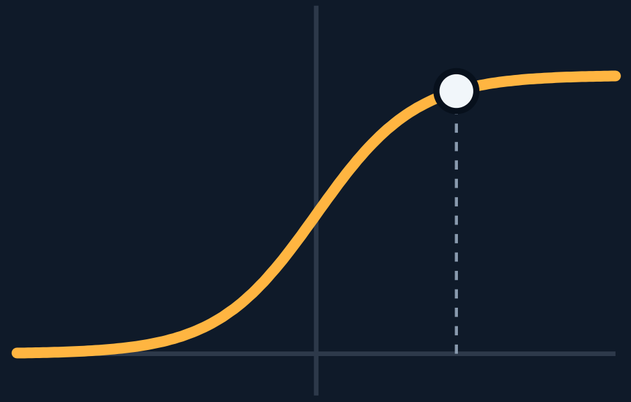
Escalón · un interruptor



Solo 0 o 1: sí o no.

1

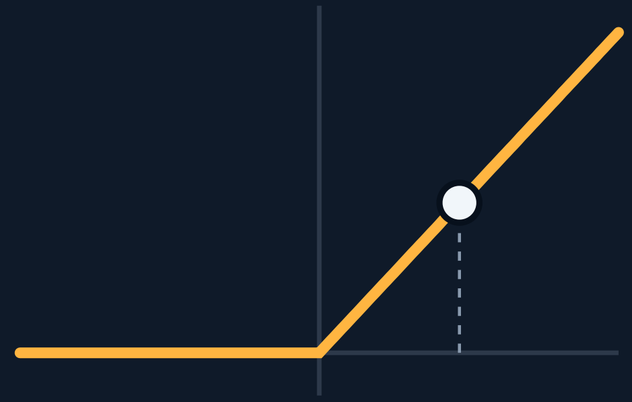
Sigmoide · un regulador de luz



Entre 0 y 1: una probabilidad.

0.943

ReLU · una válvula



Si es negativo, 0; si no, pasa igual.

2.81

puntaje z

2.81

socio A · z = 2.81

z = 0

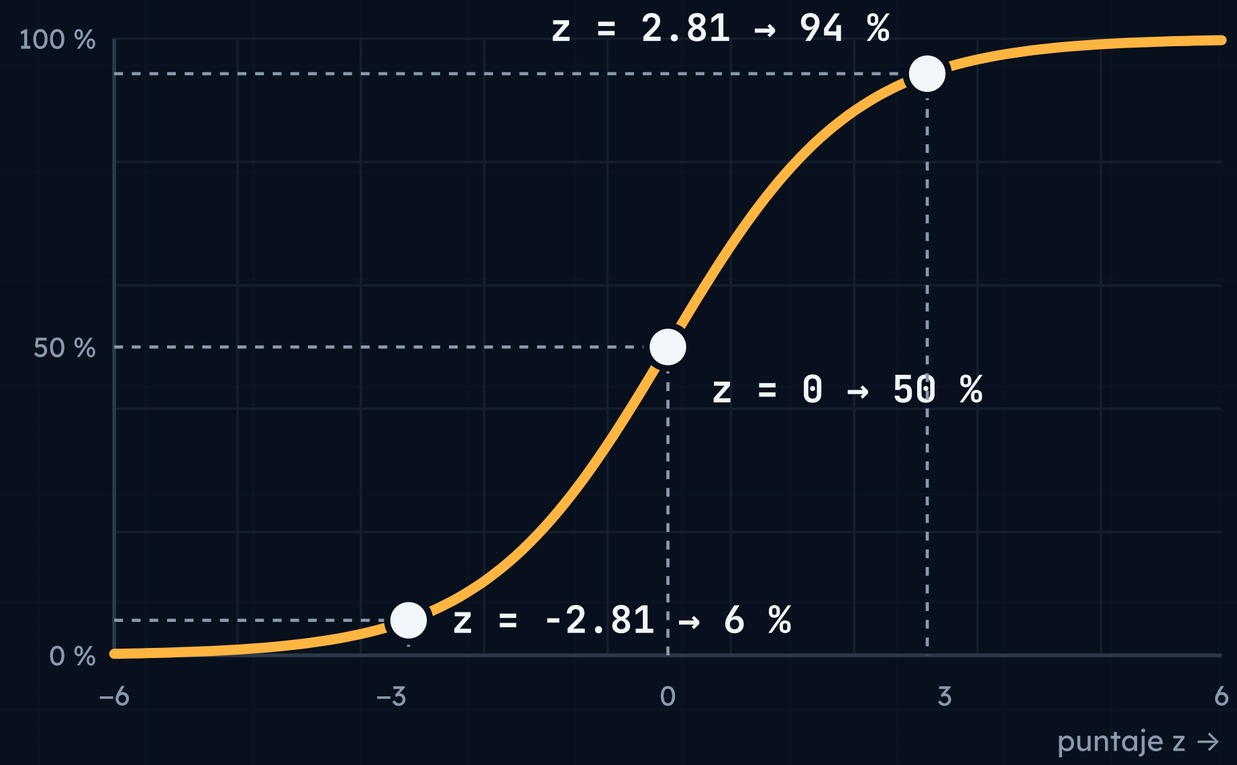
z = −2.81

De puntaje a probabilidad: la sigmoide

1. Puntaje muy negativo → **casi 0 %**
2. Puntaje 0 → **50 %**: la neurona duda
- **A** El socio A, z = 2.81 → **94 % de que paga**

$$\underbrace{\sigma(z)}_{\text{probabilidad, de 0 a 1}} = \underbrace{\frac{1}{1+e^{-z}}}_{\text{aplasta cualquier puntaje}}$$

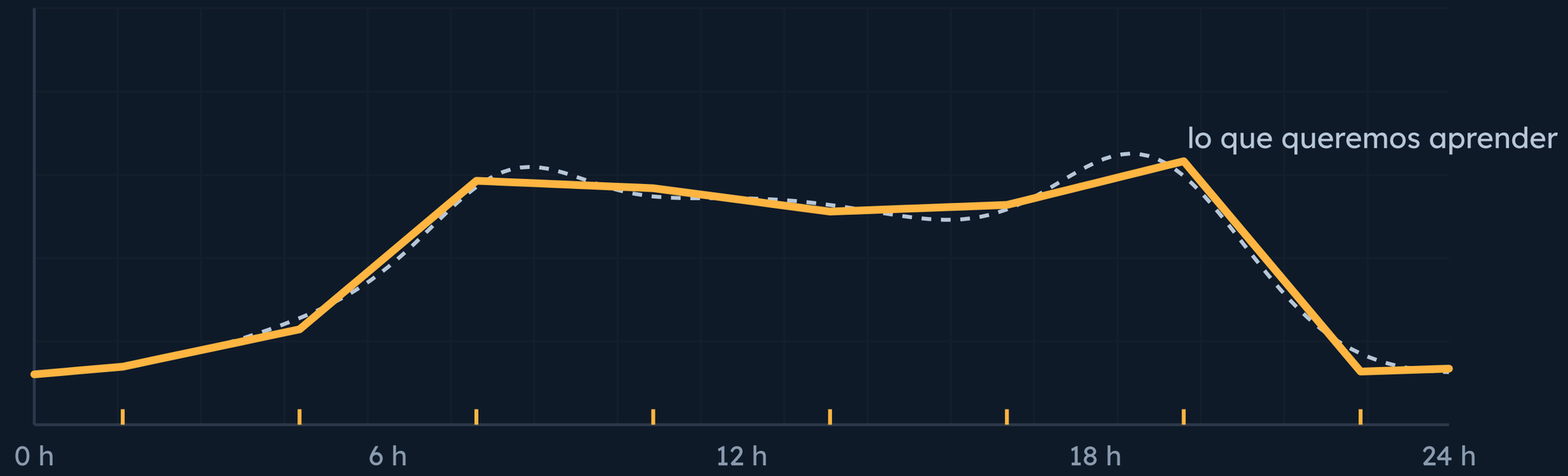
Léelo así: si z es muy grande, e^{-z} casi desaparece y queda 1/1 = 1; si z es muy negativo, el de abajo crece y queda casi 0.



Sin activación, apilar capas no sirve de nada

Idea clave 2

Una suma de sumas sigue siendo una línea. La activación **dobla** la línea, y con varios dobleces se arma cualquier curva.



Sin activación | 2 neuronas ReLU | 4 | 8 | 16

8 dobleces · error 0.0005

Curva ilustrativa, por ejemplo la demanda de taxis a lo largo del día.

03

Cómo aprende una neurona

Equivocarse, corregir un poquito y repetir.

- El ciclo
- La regla
- Una corrección
- Míralo aprender
- El límite: XOR

Viernes · 15 minutos

Aprender es equivocarse y corregir

Todo el machine learning, del perceptrón a ChatGPT, es este ciclo.

Como sazonar un locro: pruebas, notas que le falta sal, echas una pizca y vuelves a probar. Una pizca, no medio salero.

Una vuelta completa por todos los ejemplos se llama **época**.



La regla del perceptrón: corregir solo cuando falla

Si se equivocó, empuja los pesos hacia la respuesta correcta. Si acertó, no toca nada.

$$w_{\text{nuevo}} = w + \eta \cdot y \cdot x$$

w_{nuevo}	w	η	y	x
peso nuevo	peso viejo	un pasito	lo correcto: +1 o −1	el dato

Léelo así: el peso nuevo es el viejo más un pasito **hacia la respuesta correcta**. El sesgo igual: b + η·y.

1958 Frank Rosenblatt presenta el perceptrón

Debía decir sí · y = +1

Le **suma** el dato a los pesos: la próxima vez ese socio tendrá más puntaje.

Debía decir no · y = −1

Le **resta** el dato: la próxima vez tendrá menos puntaje.

Acertó

No cambia nada. Solo aprende de sus errores.

Una corrección, con números

Un socio con x = (1.0, 1.0) que **sí pagó** (y = +1). La neurona empieza con w = (0.2, −0.5) y b = 0.

Antes

$$z = 0.2 - 0.5 + 0$$

$$-0.30$$

Rechaza al socio. **Se equivocó.**

Corrige (η = 1)

$$w_1 = 0.2 + 1 = 1.2$$

$$w_2 = -0.5 + 1 = 0.5$$

$$b = 0 + 1 = 1$$

Le suma el dato a cada peso y 1 al sesgo.

Después

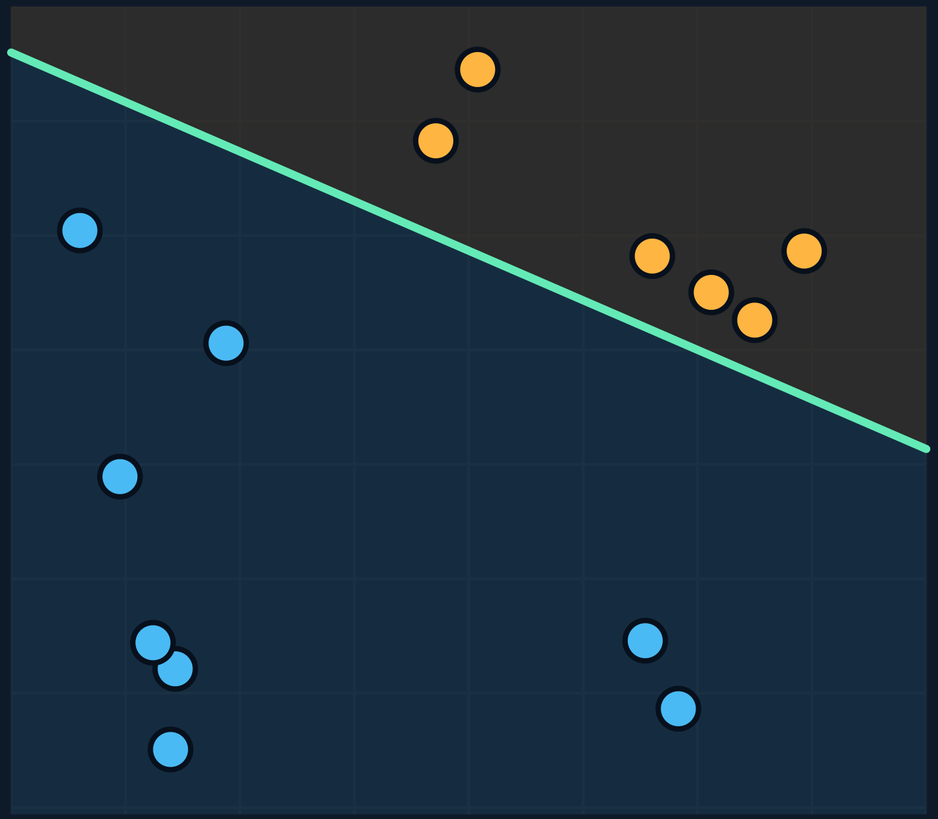
$$z = 1.2 + 0.5 + 1$$

$$+2.70$$

Ahora aprueba. **Acertó.**

Mira cómo aprende, error por error

Recorre los puntos uno por uno; cada vez que se equivoca, mueve la línea. Si existe una línea que separa los datos, la encuentra.



¡Aprendió! 0 errores

1. Vuelta 1: **5 errores**
2. Vuelta 2: **0 errores**

▶ **Entrenar**

Un paso

Reiniciar

El primer límite: la luz de la escalera

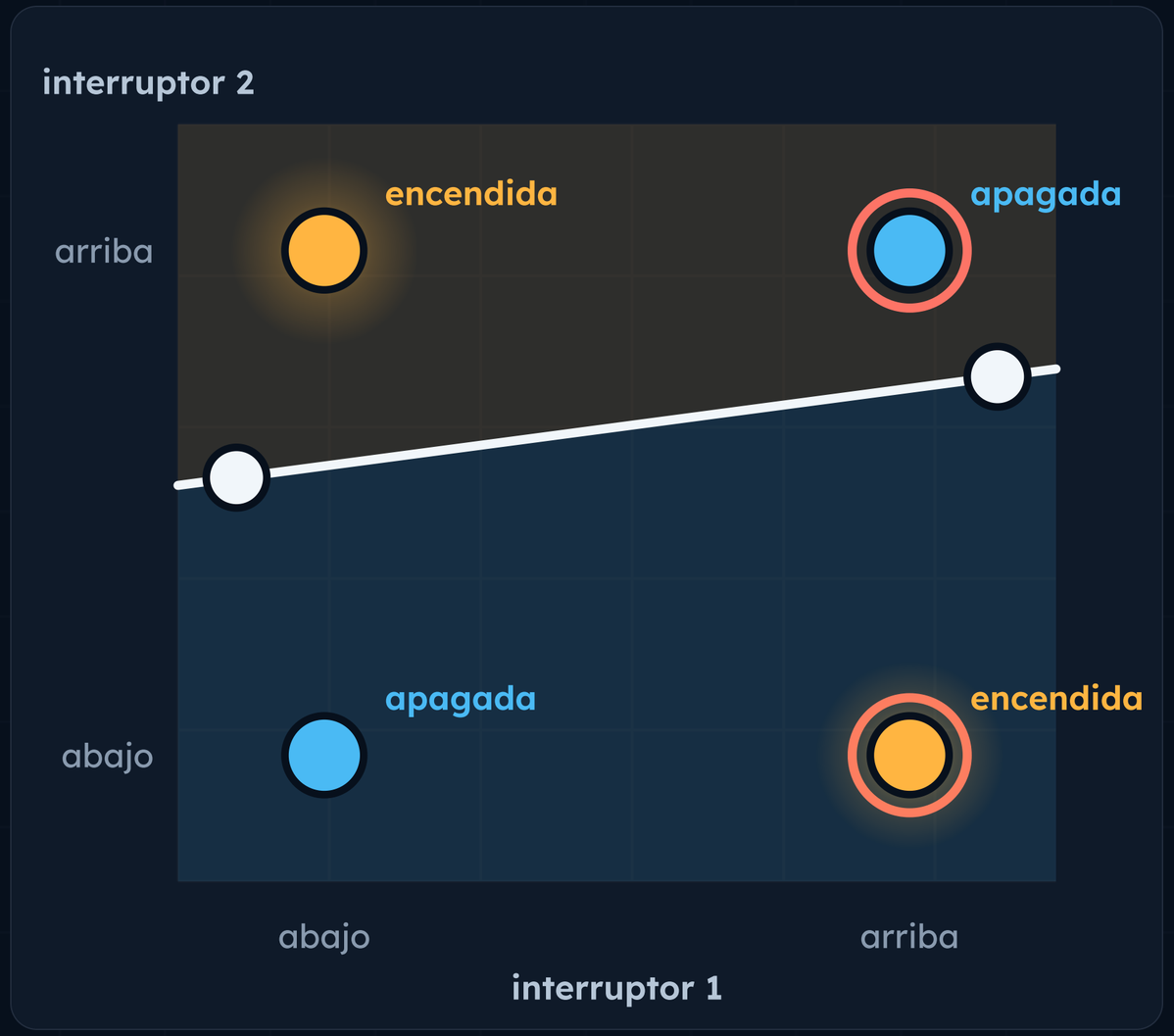
Idea clave 3

Con dos interruptores, la luz se enciende si están en posiciones **distintas**. Eso es XOR.

Arrastra los extremos de la línea

No se puede. Una neurona solo traza una recta, y ninguna recta separa XOR.

1969 Minsky y Papert lo demuestran: llega el primer «invierno» de la IA.



Bien separados: 2 de 4

Probar 1 000 líneas al azar

La neurona en Python: lo que vas a escribir

Cinco líneas de código hacen la cuenta de Ana. El NB01 te lleva por cada una, paso a paso.

```python
import numpy as np                      # math on whole lists

x_ana  = np.array([80, 70, 90, 60]) # data: Ana's grades
w_nota = np.array([.8/3]*3 + [.2])  # weights
b_nota = -70                        # bias: the bar

z = w_nota @ x_ana + b_nota         # pairs times, summed
print(z, z ⩾ 0)

sigmoid = lambda z: 1/(1 + np.exp(-z))  # z → 0..1
```

```
salida  6.0 True
```

`import` trae una librería. **NumPy** es como una hoja de cálculo muy rápida dentro de Python.

`np.array` guarda una lista de números para hacer cuentas con todos a la vez. Datos, pesos y sesgo: la neurona del bloque 02.

`@` multiplica cada nota por su peso y suma todo (el **producto punto**). Esa línea **es** la neurona.

`lambda` define una función corta. Esta es la sigmoide: el regulador de luz.

¿Python oxidado? La **Parte 0** del NB01 repasa variables, listas, funciones, NumPy y gráficas en 15 minutos.

Práctica en clase: NB01, secciones 1 a 5

En clase · sin nota 20 minutos

Lo que acabamos de ver, ahora con código explicado paso a paso. Cada tarea te dice ✅ correcto, ❌ todavía no (con una pista) o ⏳ falta.

Qué deberías ver

- **Parte 0** (si la necesitas) · Python con tu nota: Beto da **62**
- §1 · Ana: **z = +6** · el socio A: **z = 2.81**
- §2 · Sigmoide ≈ **0.943**: 94 % de que paga
- §3 · Una corrección: **−0.30 → +2.70**
- §4 · El perceptrón aprende AND: **0 errores**
- §5 · Con XOR **nunca** deja de equivocarse

deeplearningweek1.vercel.app/?nb01

Archivo → **Guardar una copia en Drive**
y ejecuta primero **Configuración**

04

Medir el error y mejorar

Bajar la montaña con neblina, paso a paso.

- La pérdida
- Castigar la falsa seguridad
- El Cotopaxi
- La pendiente
- El tamaño del paso

Viernes · 25 minutos

La pérdida: un número que dice qué tan mal vamos

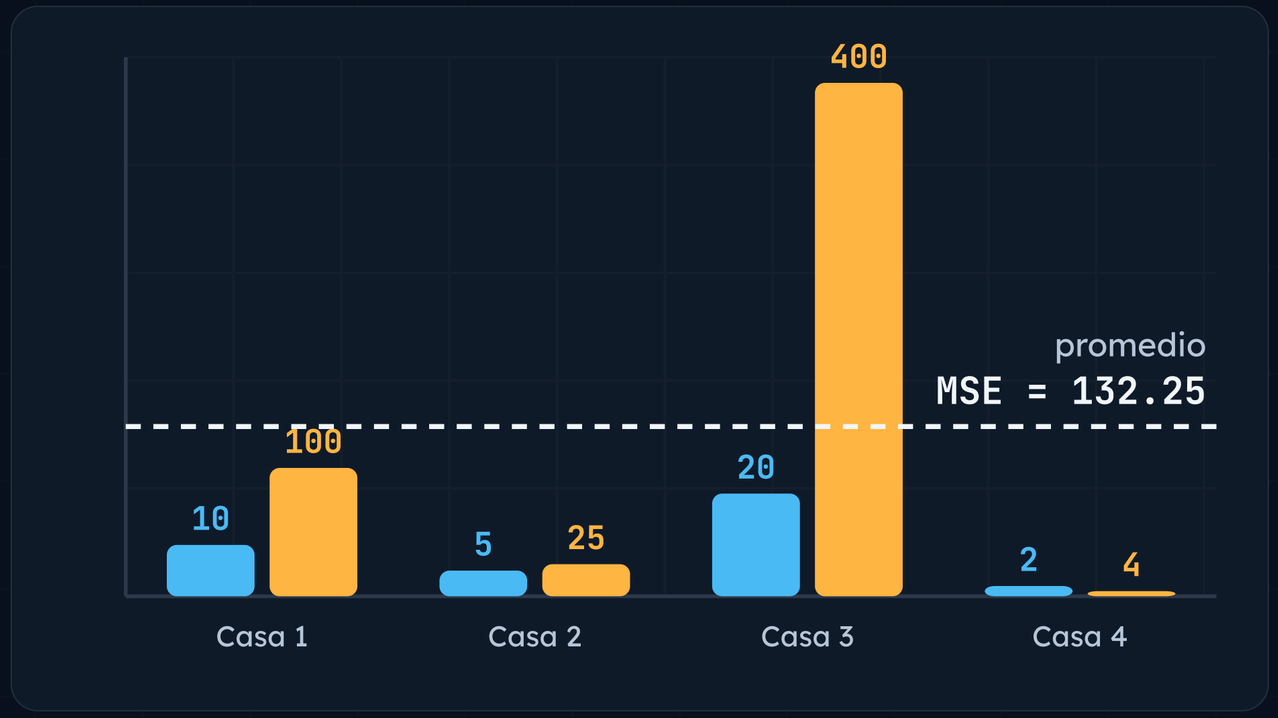
Como una nota al revés: **0 es perfecto**.

El modelo predice el precio de cuatro casas en Quito y se equivoca por 10, 5, 20 y 2 mil dólares.

- el error de cada casa
- el error al cuadrado: **la casa 3 pesa casi todo**

$$\underbrace{\text{MSE}}_{\text{qué tan mal vamos}} = \text{promedio de } \underbrace{(\hat{y} - y)^2}_{\text{cada error, al cuadrado}}$$

Léelo así: (100 + 25 + 400 + 4) ÷ 4 = **132.25**. Legendre ya lo usaba en 1805.



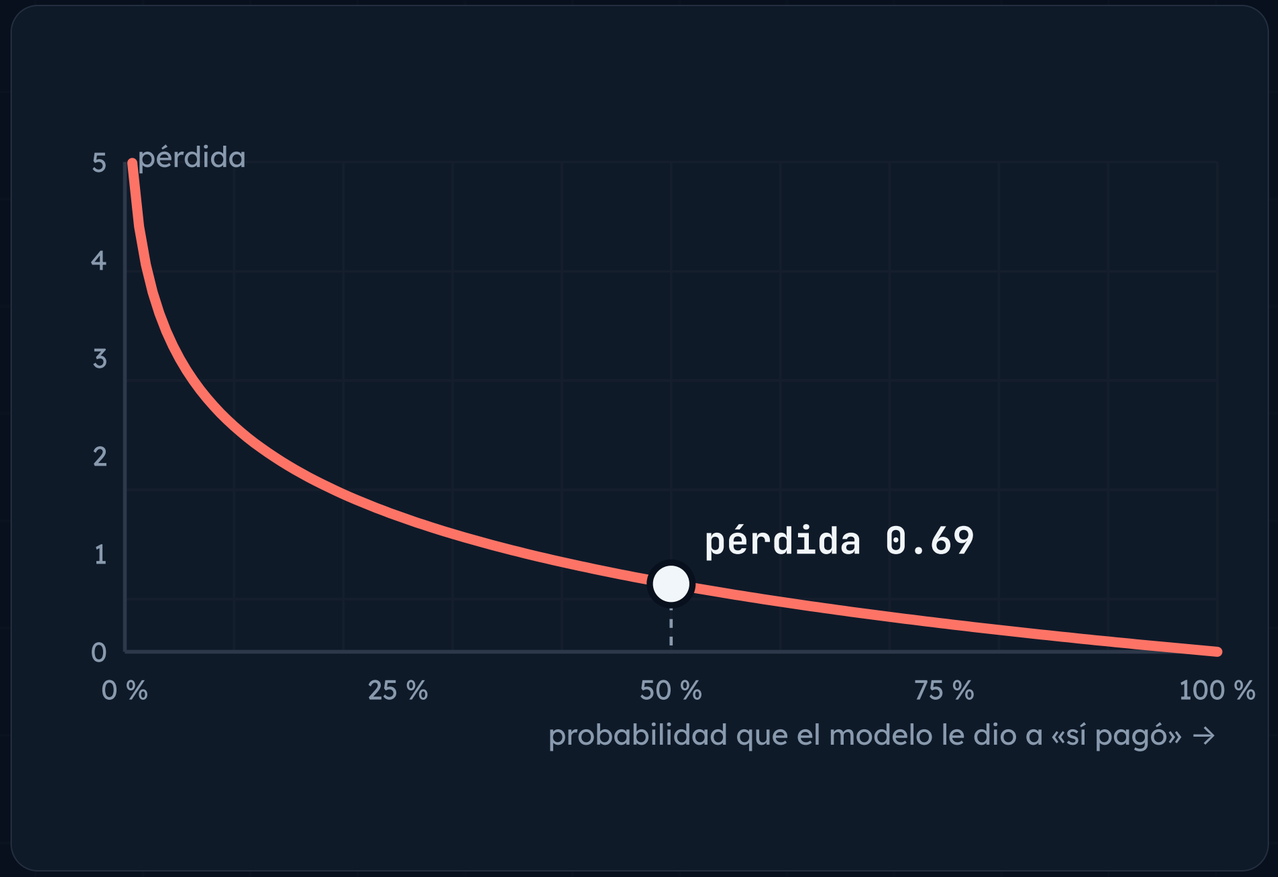
Para clasificar: castigar la falsa seguridad

El socio **sí pagó**. ¿Cuánto castigamos al modelo según lo que dijo?

Como al meteorólogo: si dijo «99 % sin lluvia» y llovió, pierde toda credibilidad.

$$\underbrace{\text{pérdida}}_{\text{el castigo}} = -\log \underbrace{(p)}_{\substack{\text{cuánto apostó} \\ \text{por lo que pasó}}}$$

Léelo así: apostó 99 % y acertó: castigo 0.01. Apostó 1 %: castigo **4.61**. En Keras se llama `binary_crossentropy`.



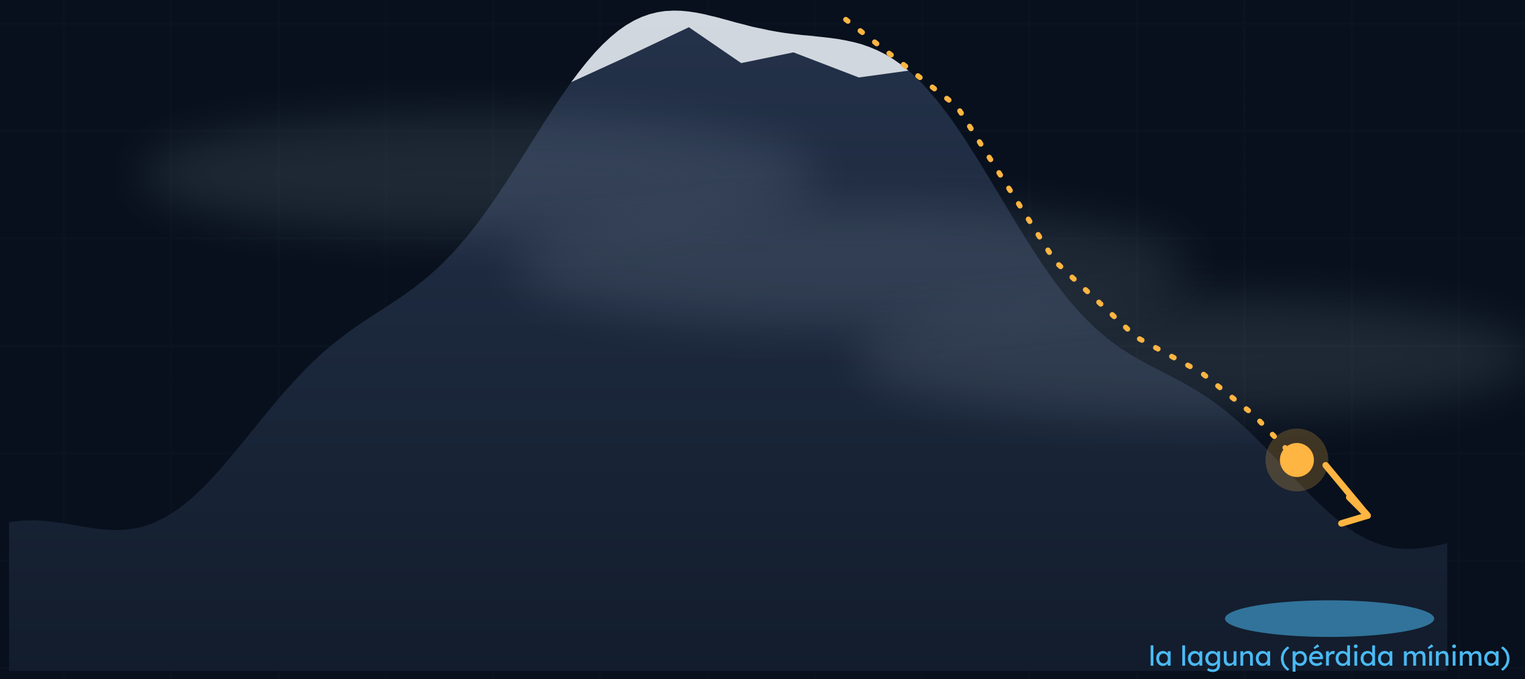
el modelo dijo 50 %

99 % **50 %** **1 %**

Dudó: castigo moderado.

Entrenar es bajar el Cotopaxi con neblina

No ves la laguna, pero sientes con el pie hacia dónde baja el terreno. Das un paso. Vuelves a tantear.



La altura
= la pérdida

Dónde estás
= los pesos

La inclinación bajo tus pies
= el gradiente

El largo de tu paso
= η, la tasa de aprendizaje

La pendiente dice hacia dónde bajar

Idea clave 4

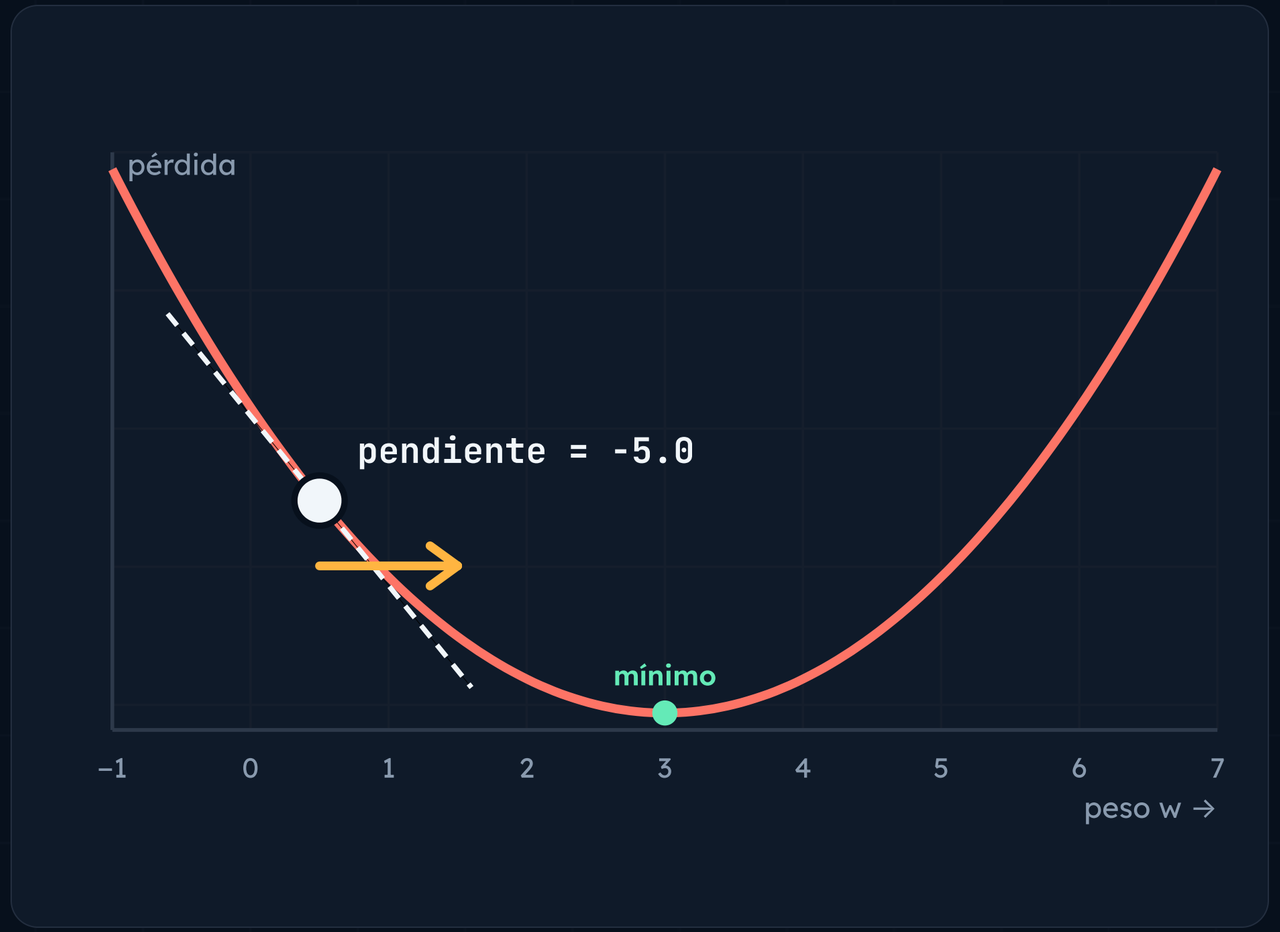
Siempre damos el paso **en contra** de la pendiente: así la pérdida baja.

$$w_{\text{nuevo}} = w - \eta \cdot \text{pendiente}$$

- w_{nuevo}: peso nuevo
- w: donde estás
- η: el paso
- pendiente: hacia dónde sube

Léelo así: el menos significa **en contra**: restar una pendiente negativa es sumar y el peso sube. Arrastra el punto y compruébalo.

1847 Augustin Cauchy propone bajar por la pendiente



peso w — 0.50

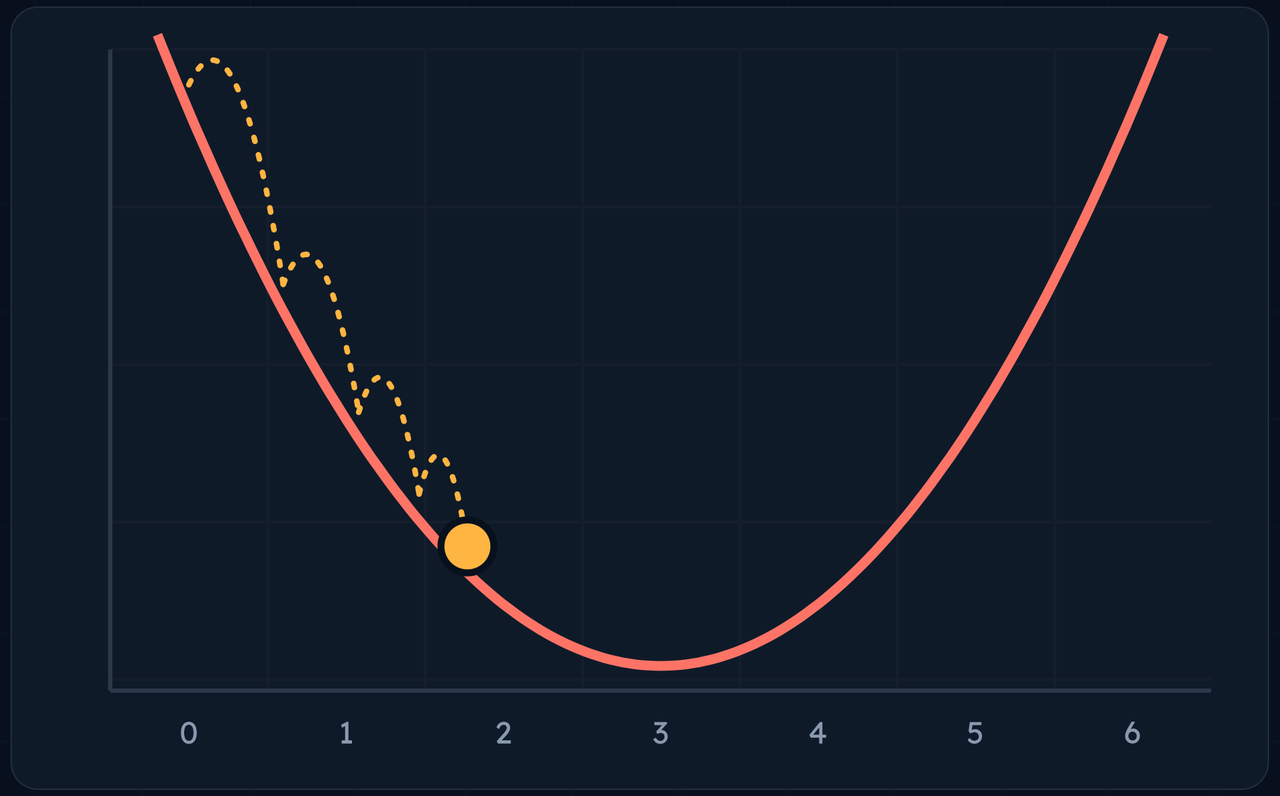
Pendiente **negativa** → el peso **sube** (paso a la derecha)

Bajando paso a paso, con números

Pérdida $(w - 3)^2$, empezando en $w = 0$, con $\eta = 0.1$.

paso	w	pendiente	pérdida
0	0.00	-6.00	9.00
1	0.60	-4.80	5.76
2	1.08	-3.84	3.69
3	1.46	-3.07	2.36
4	1.77	-2.46	1.51

La pérdida baja en cada paso y **los pasos se acortan solos**: cerca del fondo la pendiente es menor.



El largo del paso η: ni muy corto ni muy largo

Como la ducha: si giras la llave de golpe pasas de helada a hirviendo, y al corregir de golpe vuelves a helada. Prueba los tres botones.



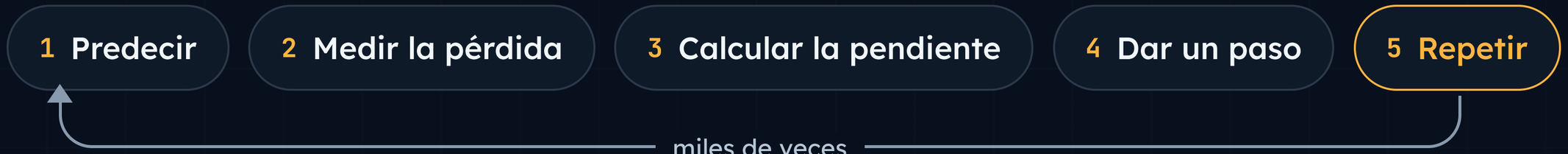


tamaño de paso η

0.40

muy corto · 0.02 | bien · 0.4 | muy largo · 1.02

La receta de todo el deep learning



Así aprende el perceptrón, una red de tres capas y también un modelo de lenguaje con miles de millones de pesos.

Práctica en clase: NB01, secciones 6 a 8

En clase · sin nota · 10 minutos

La pérdida, el descenso de gradiente y el tamaño del paso, con código. Las tareas finales quedan para la casa.

Qué deberías ver

- §6 · MSE = **132.25**; BCE: **0.01 · 0.69 · 4.61**
- §7 · Tu descenso llega a **w ≈ −2**
- §8 · Compara η: lento, bien y diverge
- **Tareas finales (en casa)** · la nota de todo el grupo, elige tu η y el Mini-reto 1

deeplearningweek1.vercel.app/?nb01

Archivo → **Guardar una copia en Drive** y ejecuta primero **Configuración**

Lectura de la semana: «Más allá de Adam»

Lectura de tendencia

Hoy bajamos la montaña con pasos de tamaño fijo. Los optimizadores modernos ajustan el paso solos. ¿Vale más un mejor algoritmo o más cómputo?

2024→2025

Muon, un optimizador nuevo, pasa de récords en competencias abiertas a entrenar **Kimi K2**, un modelo de un billón de parámetros.

Jordan (s. f.); Kimi Team (2025)

1.4× → 1.1×

Con una carrera justa para todos, la ventaja se encoge: grande en modelos pequeños, modesta en los grandes.

Wen, Hall, Ma y Liang (2025)

>3× al año

Crece la capacidad mundial de cómputo dedicada a la IA. Un optimizador mejor ayuda, pero no cambia el orden de magnitud.

Stanford HAI, AI Index (2026)

Lee la lectura y responde la encuesta en Canvas. **No hay respuestas correctas**: queremos tu opinión. Mañana, en el bloque 07, hacemos correr a los optimizadores.

Lo que te llevas hoy

Una neurona	**suma con pesos y sesgo, y traza una línea**
La activación	**dobla la línea y da probabilidades**
Aprender	**es bajar la montaña: pendiente y paso**

Mañana: ¿cómo vencer a la luz de la escalera (XOR)? Juntando neuronas en capas.

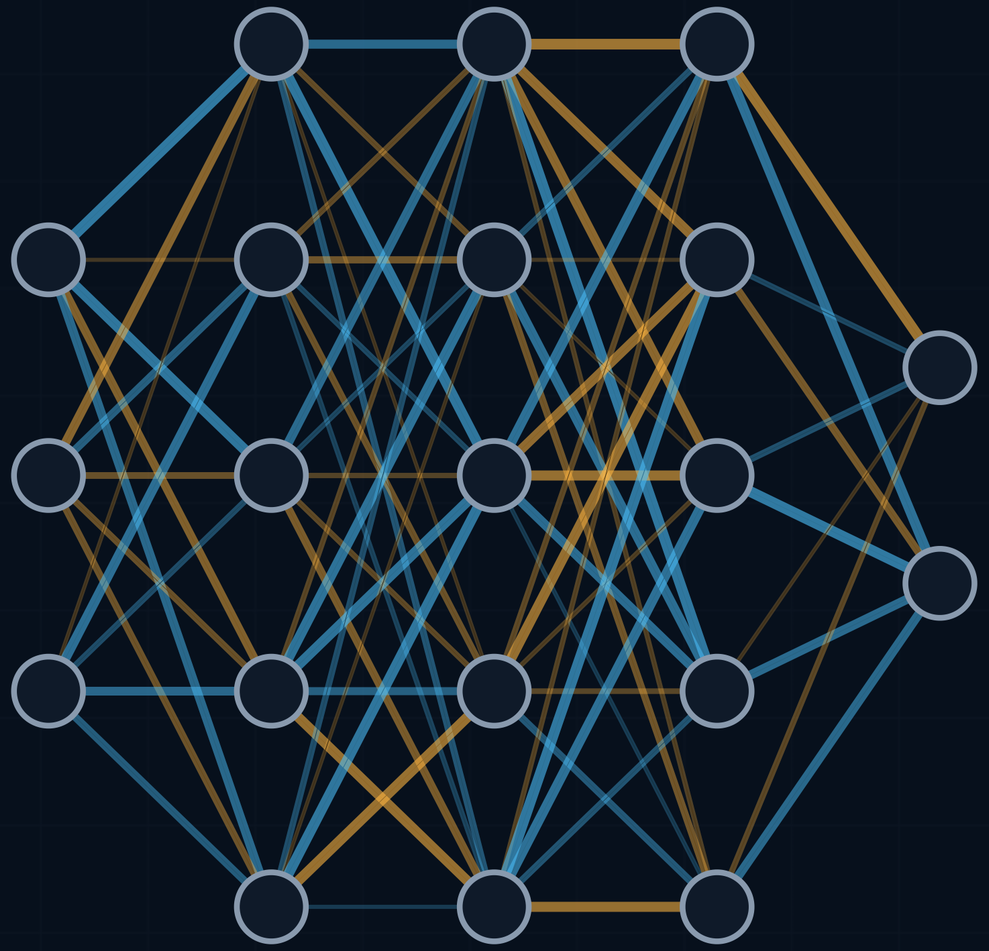
En el chat, del 1 al 5: **¿podrías explicar el descenso de gradiente a un colega?**

Muchas neuronas, una red

Hoy vencemos a XOR, repartimos la culpa y entrenamos con Keras.

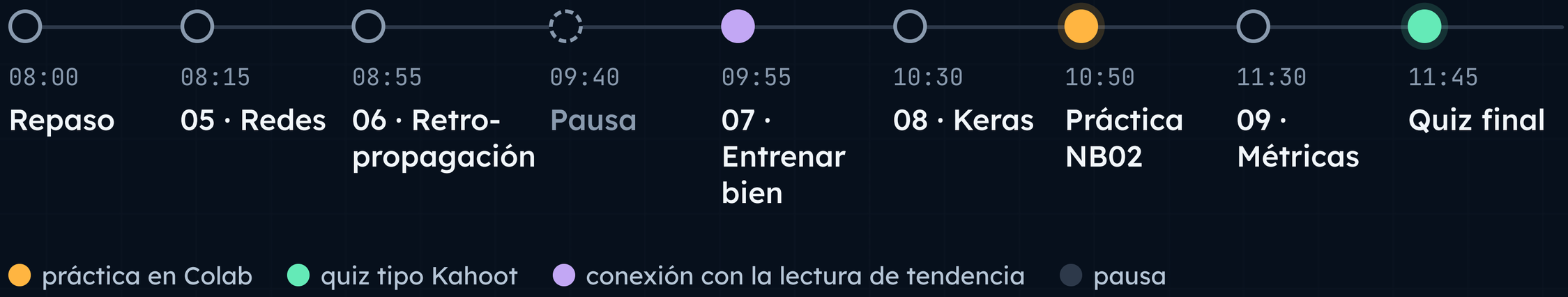
Semana 1 · Sábado 24

08:00–12:00



El sábado en nueve paradas

De la red a Keras, con una práctica larga en Colab y el quiz final para cerrar.



El viernes en cuatro ideas

¿Quién explica cada una con sus palabras?



Una neurona
suma con pesos y sesgo, y traza una línea



La activación
dobla la línea y da probabilidades



El perceptrón
aprende de sus errores… pero no puede con XOR



El descenso de gradiente
baja la montaña paso a paso

Pregunta para el chat: ¿qué problema nos quedó pendiente? Pista: la luz de la escalera.

05

De una neurona a una red

Muchas líneas simples, combinadas, forman decisiones complejas.

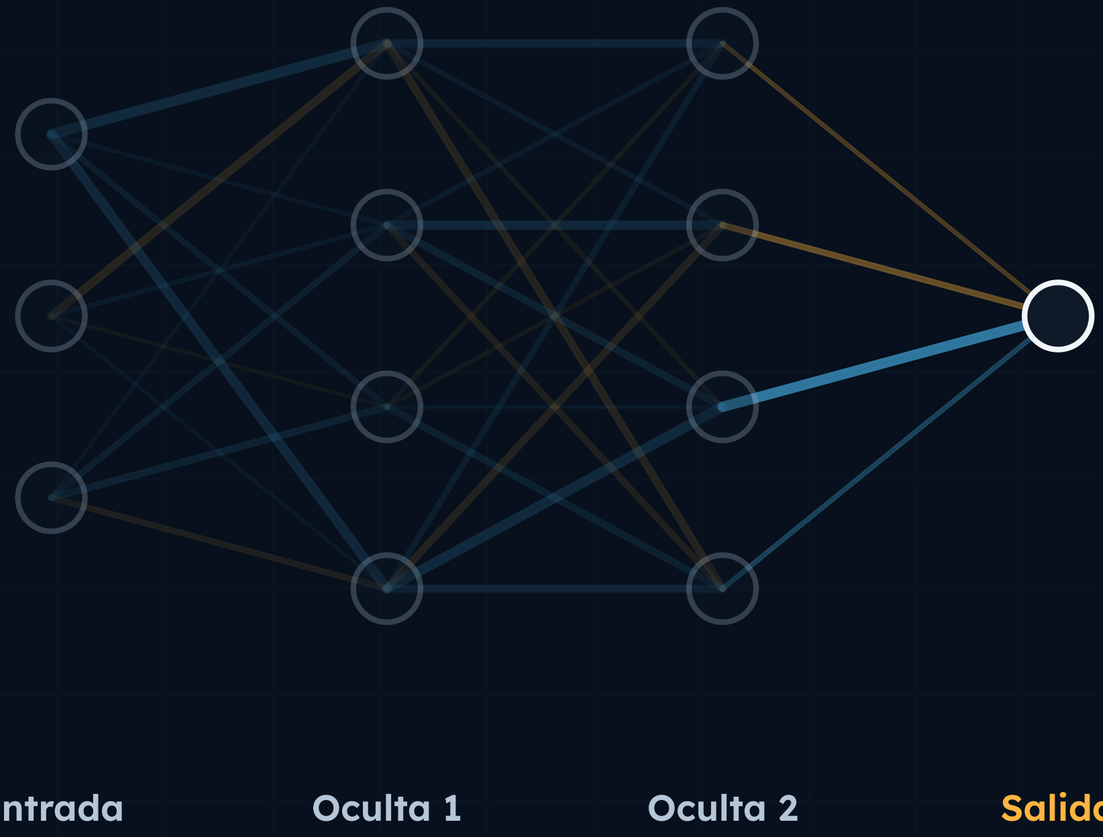
- Capas
- XOR resuelto
- Pistas cada vez más ricas
- Juega a la red
- Contar parámetros

Sábado · 40 minutos

Una red son neuronas organizadas en capas

1. **Entrada:** los datos. No calcula nada.
2. **Ocultas:** inventan sus propias pistas a partir de los datos.
3. **Salida:** toma la decisión final.

Como una cooperativa con agencias en **Riobamba, Latacunga y Ambato**: cada agencia resume lo que ve y el comité central decide con esos resúmenes.

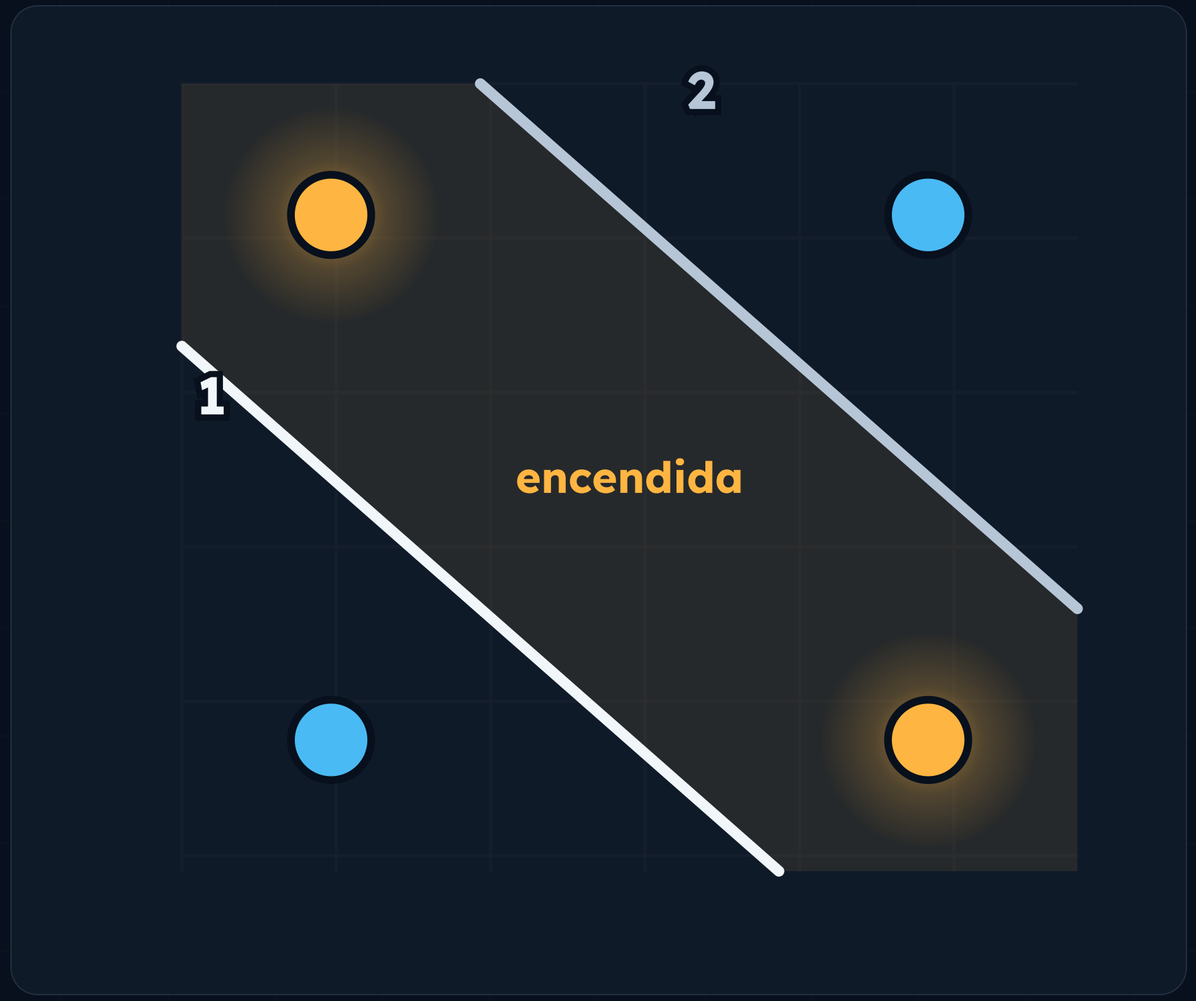


Dos neuronas vencen a la luz de la escalera

Idea clave 3

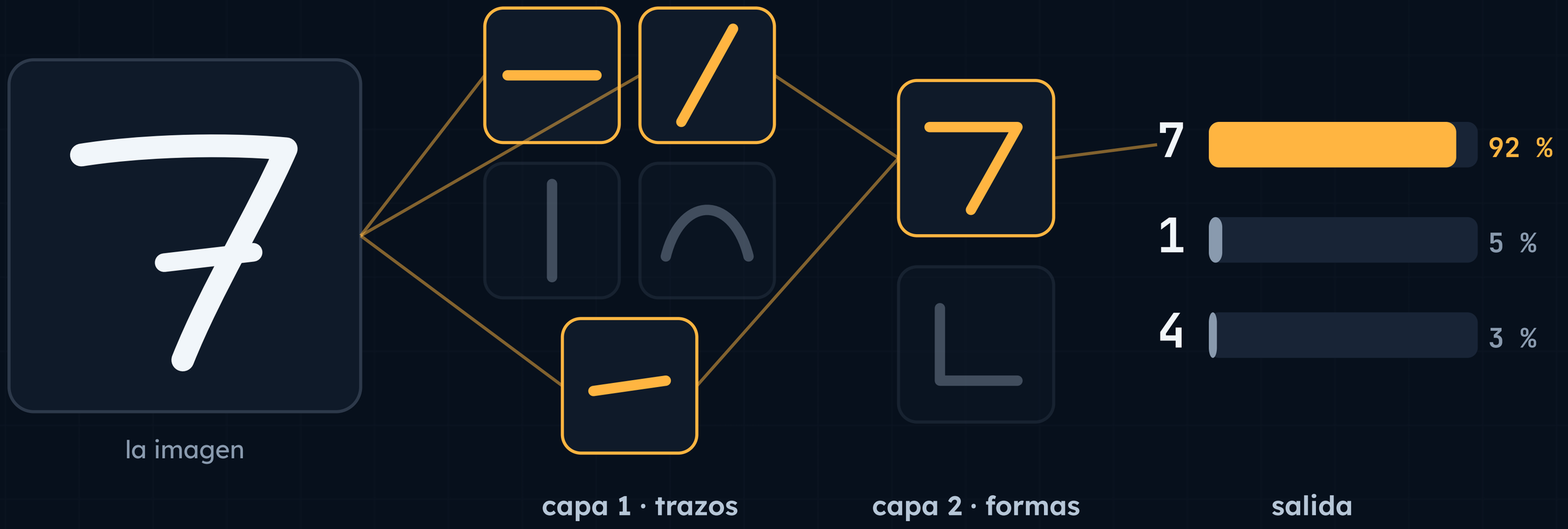
Cada neurona oculta traza su línea. La salida combina las dos.

1. Una aprende: **¿al menos un interruptor arriba?**
2. Otra aprende: **¿los dos arriba?**
3. = Luz encendida si la primera dice sí y la segunda dice no: **la franja del medio**.



Cada capa arma pistas más elaboradas

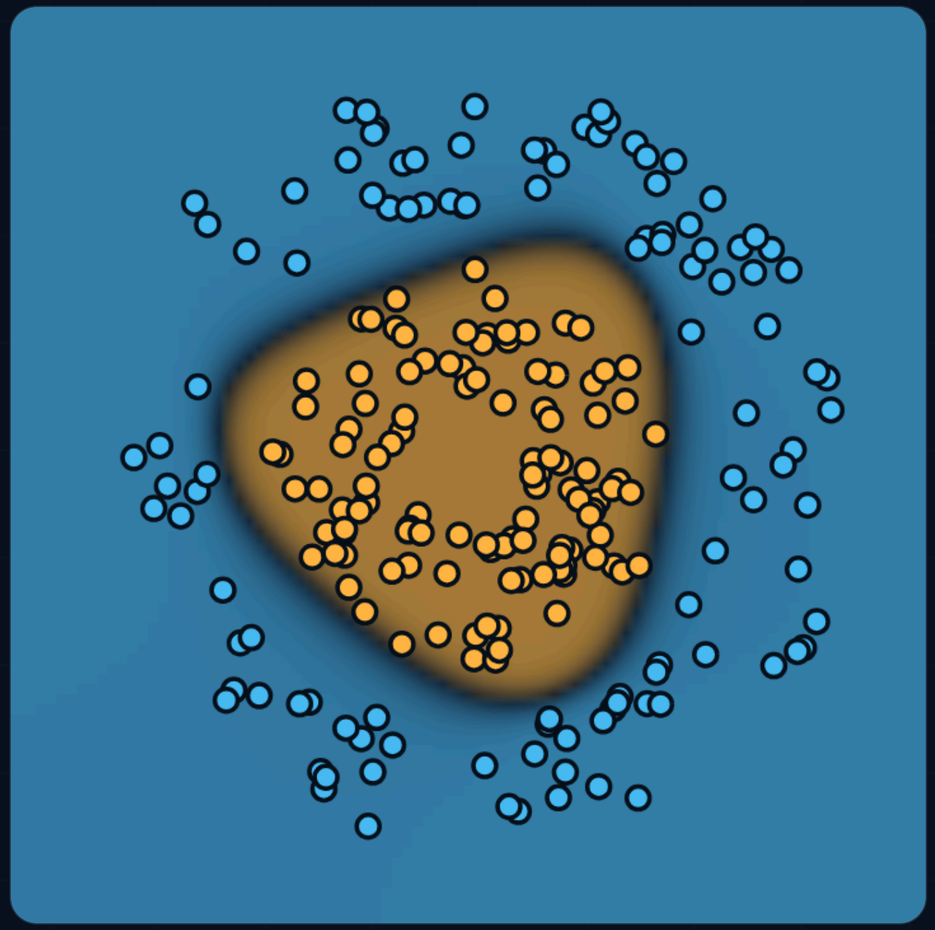
Leer un 7 escrito a mano, como el monto de una papeleta de depósito.



Esquema ilustrativo: en una red real las pistas no son tan limpias.

Juega: ¿cuántas neuronas necesita?

Una red pequeña que entrena aquí mismo. Elige los datos y el número de neuronas ocultas.



Datos

XOR | Círculos | Lunas

Neuronas en la capa oculta

1 | 2 | 4 | 8

▶ Entrenar ↺

época	600
pérdida	0.021
exactitud	100 %
números que aprende	17

¿Cuántos números aprende una red?

Como las perillas de una consola de sonido: un peso por cada cable y un sesgo por cada neurona. Entrenar es girarlas todas hasta que suene bien.



$3 \times 4 + 4 = 16$

$4 \times 1 + 1 = 5$

21 números que aprender

La red de las lunas del NB02: **337**

Un modelo de lenguaje actual: **cientos de miles de millones**

06

Retropropagación

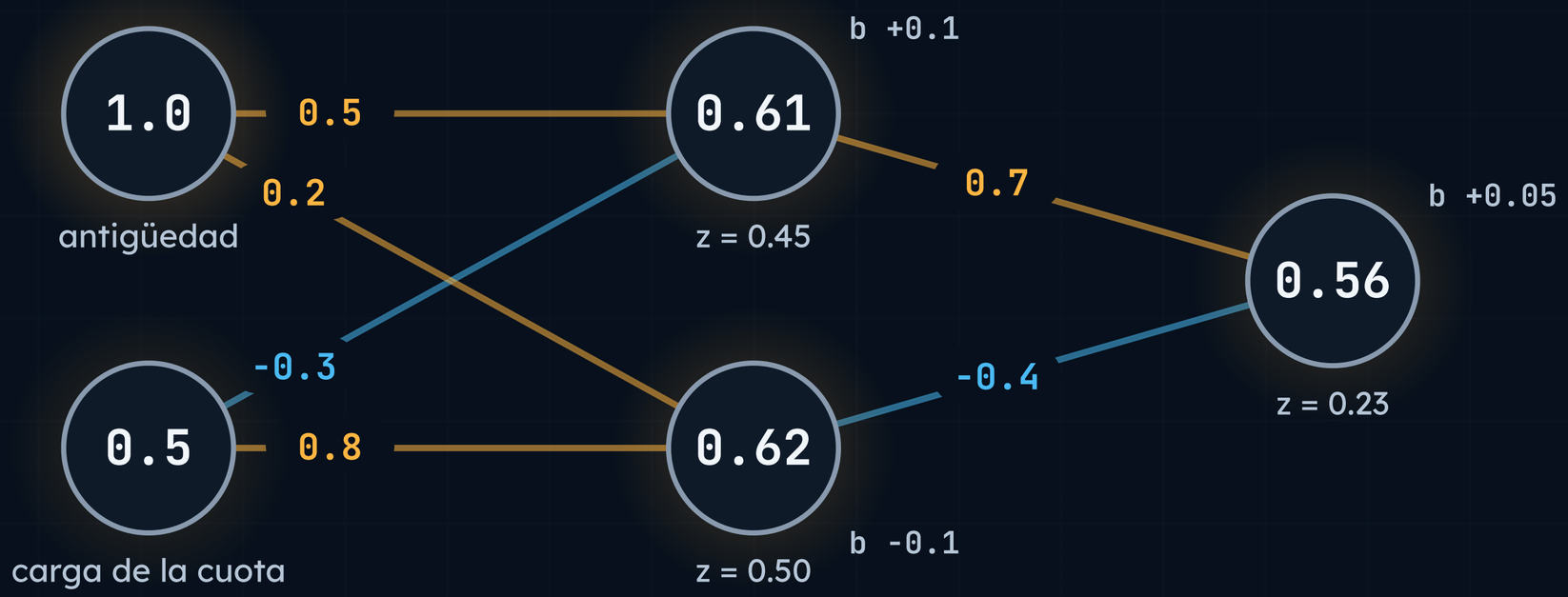
Cómo se reparte la culpa del error entre miles de pesos.

Hacia adelante · Repartir la culpa · Engranajes · Hacia atrás · Actualizar · TensorFlow

Sábado · 45 minutos

Paso 1 · Hacia adelante: la red predice

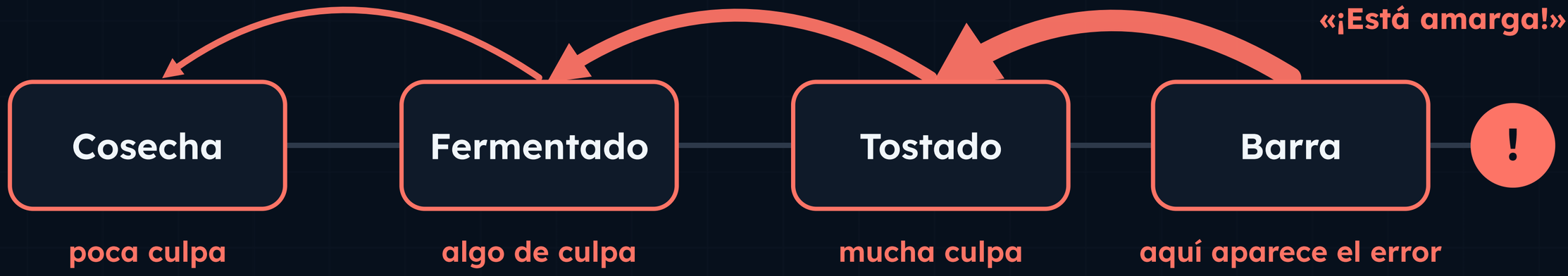
Un socio con antigüedad 1.0 y carga de la cuota 0.5 (datos escalados). Cada neurona hace la cuenta del viernes.



La red dice **56 % de que paga**. El socio sí pagó: pérdida = –log(0.56) = **0.59**.

Paso 2 · Hacia atrás: repartir la culpa

El error se descubre al final. La culpa viaja hacia atrás, según cuánto influyó cada paso.



Eso es literalmente la **retropropagación** (backpropagation): propagar la culpa hacia atrás.

1986 Rumelhart, Hinton y Williams la popularizan

La regla de la cadena: engranajes que se multiplican

Si A mueve a B tres veces más rápido, y B mueve a C seis veces más rápido, A mueve a C 3 × 6 = 18 veces más rápido.



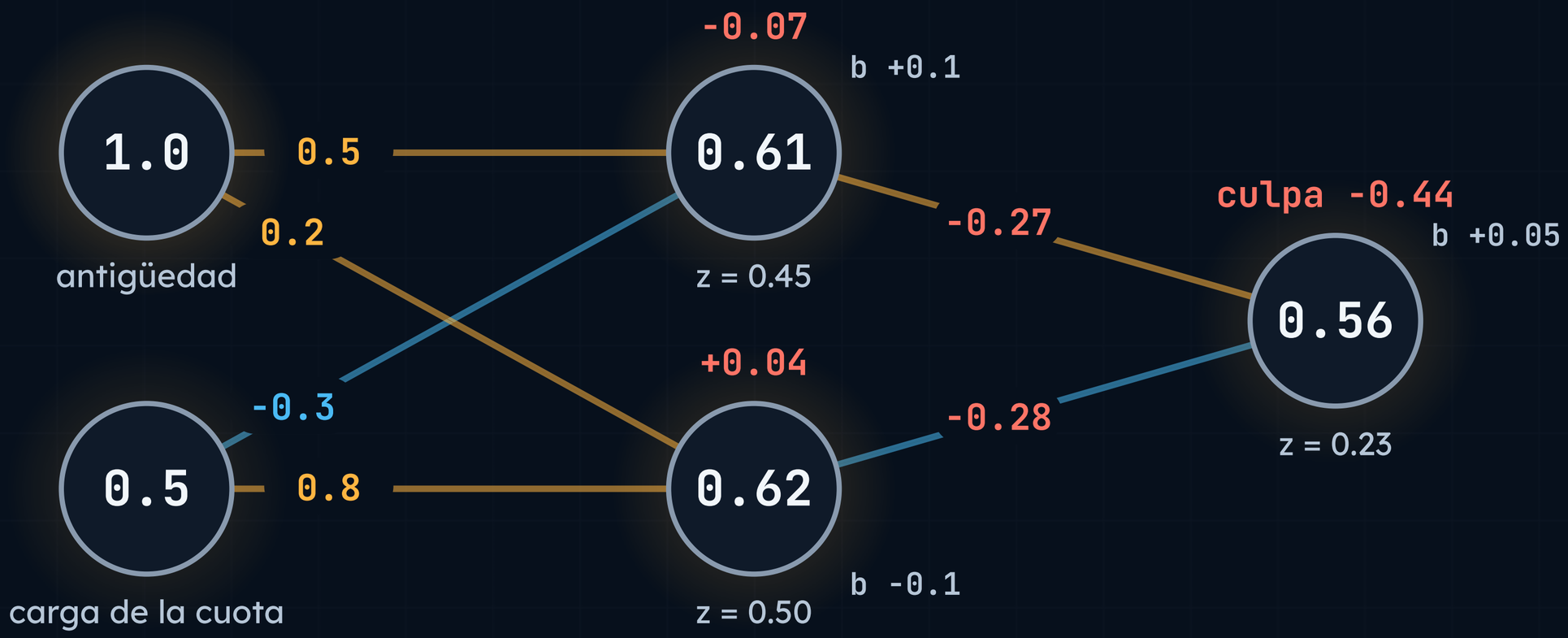
$$\underbrace{18}_{\text{A mueve a C}} = \underbrace{3}_{\text{A mueve a B}} \times \underbrace{6}_{\text{B mueve a C}}$$

Léelo así: en una red, la culpa de un peso es la culpa de la salida por lo que entró por su cable: −0.44 × 0.61 = −0.27. **Los efectos se multiplican a lo largo de la cadena.**

1676 Leibniz escribe la regla de la cadena

Hacia atrás, con los números

La misma red, pero ahora los números viajan de derecha a izquierda.



Ojo con la oculta 2: su peso hacia la salida es negativo, así que para que la salida suba, **ella debe bajar**.

Paso 3 · Actualizar: un paso cuesta abajo

A cada peso le restamos η por su culpa. Es el descenso de gradiente del viernes.



$$\underbrace{0.7}_{\text{peso viejo}} - \underbrace{0.5 \times (-0.27)}_{\eta \times \text{su culpa}} = \underbrace{0.84}_{\text{peso nuevo}}$$

…y lo mismo con **los 9 números a la vez** (en verde).

Predicción: 56 % → 65 %

Pérdida: 0.59 → 0.42

Adelante, atrás, actualizar, repetido miles de veces: eso es entrenar.

TensorFlow hace la cuenta por ti

TensorFlow es la librería de Google (2015) que hace las cuentas de las redes.
Calcula todas las culpas solo: se llama **diferenciación automática**.

```
with tf.GradientTape() as tape:
    y_hat = model(x)              # forward: the tape records it
    loss = bce(y, y_hat)
grads = tape.gradient(loss, model.trainable_variables)  # backward
```

Como una grabadora: graba cada operación de la ida…

…y la reproduce al revés aplicando la regla de la cadena.

Diferencia con nuestra cuenta a mano: 5.6×10^{-17}, es decir, cero (errores de redondeo).

07

Entrenar bien

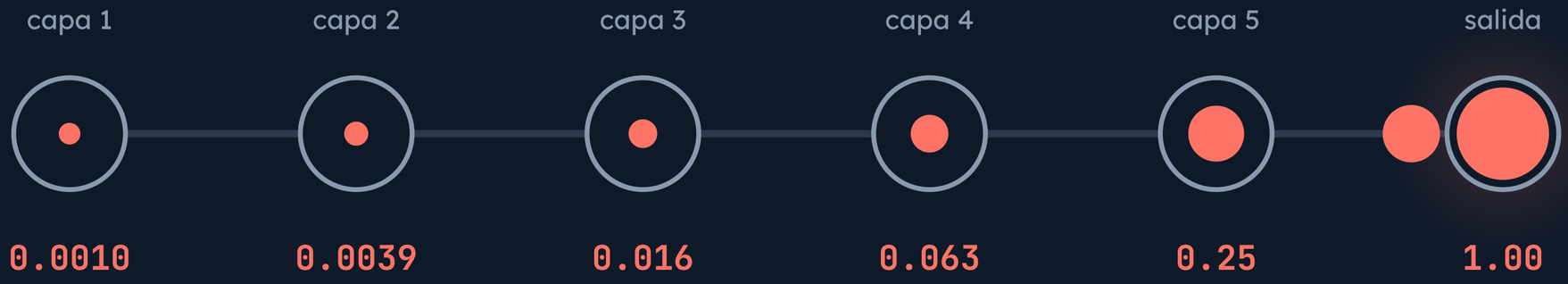
Tres trampas típicas, aprender por lotes y la carrera de los optimizadores.

- La señal que se apaga
- Neuronas clonadas
- Lotes
- La carrera
- La receta

Sábado · 35 minutos

Trampa 1: la culpa que se apaga en el camino

Como el teléfono dañado: en cada capa, la sigmoide deja pasar como mucho ¼ del mensaje y al principio ya no llega nada.



Sigmoide · ×0.25 **ReLU · ×1**

A la primera capa llega `0.001`: casi no aprende

`2010` ReLU se vuelve la activación estándar (Nair y Hinton)

Trampa 2: si todas empiezan igual, aprenden lo mismo

Como cuatro estudiantes que copian la misma tarea: cuatro cuadernos, una sola idea. Con todos los pesos en cero, las neuronas quedan clonadas.



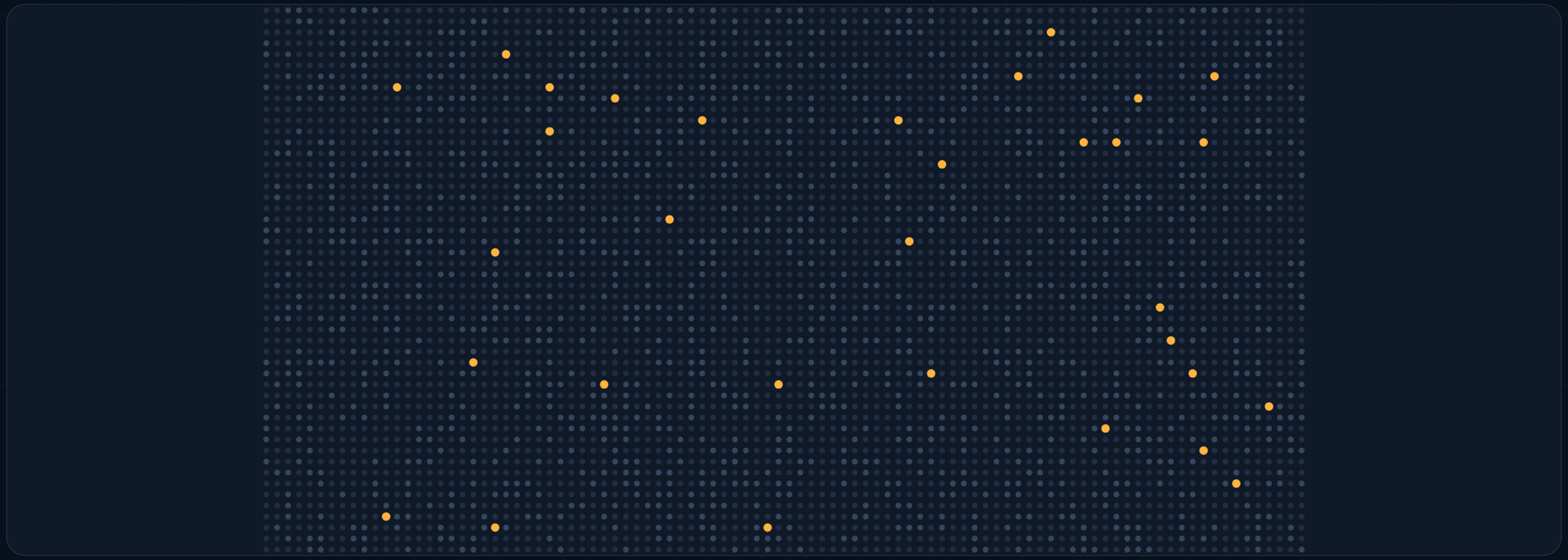
Todos los pesos en cero | **Al azar, bien escalados**

4 neuronas, **4 ideas distintas**

`2010` Glorot y Bengio: cómo sortear bien los pesos iniciales

Aprender por lotes: encuesta, no censo

Con 4 800 socios no calculamos la culpa de todos en cada paso. Tomamos 32 al azar, damos un paso y seguimos.



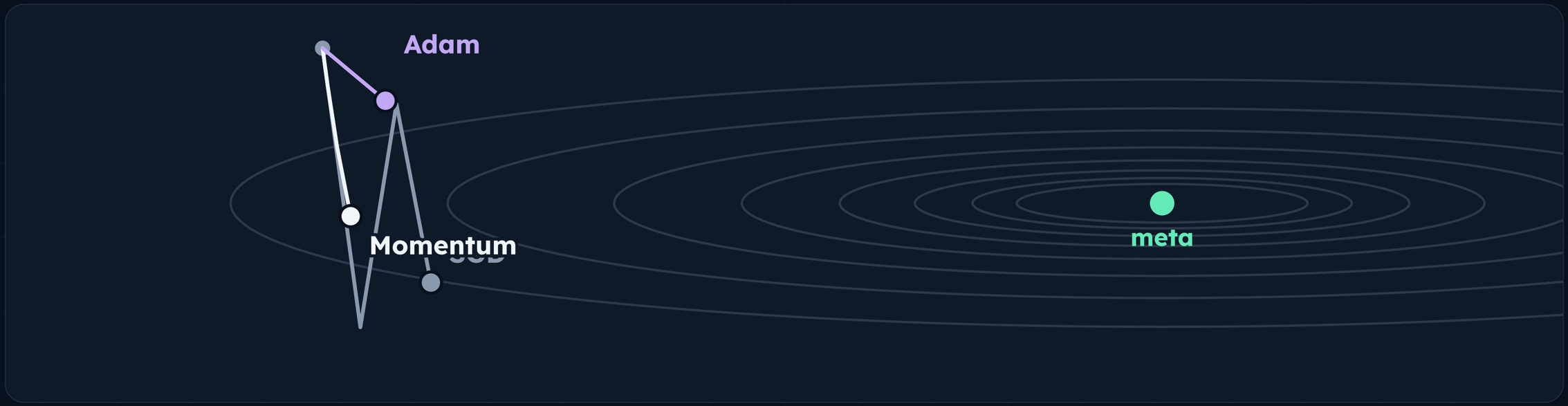
Lote 41 de 150 · 41 pasos · época 1

❚❚ Pausa

La carrera de los optimizadores

Lectura de tendencia

Mismo valle largo y angosto, tres formas de caminar. ¿Quién llega primero a la meta?



SGD: en camino… Momentum: en camino… Adam: en camino…

Repetir la carrera

SGD: pasos fijos. **Momentum** (1964): una bola que rueda y gana impulso. **Adam** (2014): zapatos a la medida de cada terreno. ¿Y con una carrera **justa**?

La receta para arrancar (y que funcione)

Un punto de partida para casi cualquier problema con datos de tabla.
Tómale foto.

1. **Escala los datos**
   que todas las columnas estén en rangos parecidos
2. **ReLU en las capas ocultas**
   la culpa llega entera
3. **Sigmoide en la salida**
   si la respuesta es sí o no
4. **Adam con learning_rate = 0.001**
   un buen punto de partida
5. **Pérdida binary_crossentropy**
   la BCE del viernes
6. **Lotes de 32 y una parte para validar**
   para medir con datos que no vio

08

Keras

Tu primera red en tres pasos: construir, compilar y entrenar.

Motor y volante · Los datos · Construir · Compilar · Entrenar · Predecir · Práctica NB02

Sábado · 20 minutos y 40 de práctica

TensorFlow y Keras: el motor y el volante

Dos librerías de Python que ya vienen en Colab. Con Keras manejas; TensorFlow hace las cuentas por debajo.

Tú · Python escribes qué quieres en Colab

↓

Keras **el volante**: capas, pérdidas y `fit`

↓

TensorFlow **el motor**: tensores y culpas calculadas solas

↓

La máquina la CPU o GPU hace las multiplicaciones

```
import tensorflow as tf   # the engine
import keras              # the wheel
from keras import layers  # the pieces

print(tf.__version__, keras.__version__)
```

salida 2.x 3.x

Puedes manejar sin abrir el capó

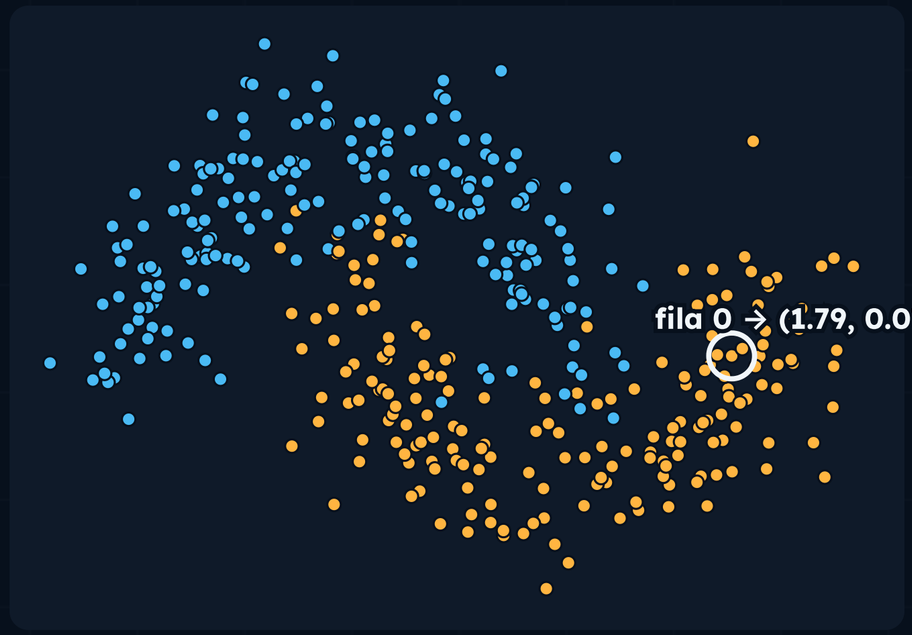
Keras te deja armar y entrenar una red en diez líneas. Saber que existe el motor ayuda cuando algo falla: por eso hicimos las cuentas a mano.

2015 nacen Keras (François Chollet, marzo) y TensorFlow (Google, noviembre)

Los datos entran como una tabla de números

Como una hoja de cálculo: cada fila es un ejemplo y cada columna un dato. En Python se llama **array**; su `shape` dice filas × columnas.

```
X_m, y_m = make_moons(600, noise=0.2, random_state=SEED)  # 600 rows, 2 columns
```



Las primeras filas de la tabla

fila	dato 1	dato 2	respuesta y
0	1.79	0.07	1
1	0.31	-0.12	1
2	2.24	0.28	1
3	-0.14	-0.12	1
4	1.63	0.30	1
5	-0.36	0.91	0

```
X.shape → (480, 2)
y.shape → (480,)
```

Construir: apilar capas como LEGO

Cada `Dense` es una capa de neuronas como las del viernes. `Sequential` las apila en orden, como piezas de LEGO.

```
model = keras.Sequential([
    keras.Input(shape=(2,)),                # 2 inputs
    layers.Dense(16, activation="relu"),    # 48
    layers.Dense(16, activation="relu"),    # 272
    layers.Dense(1, activation="sigmoid"),  # 17
])
model.summary()                             # 337
```

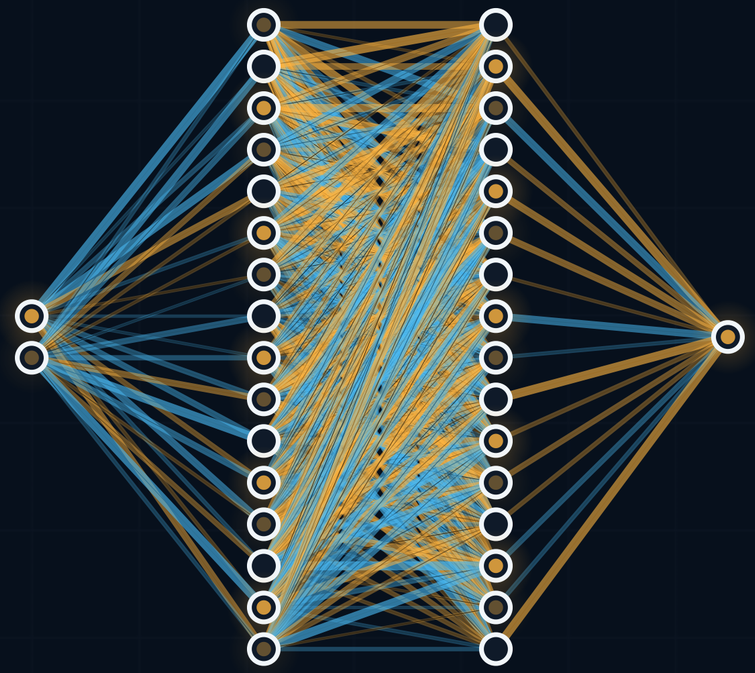
`Sequential` una pila de capas: los datos pasan en orden

`Input(shape=(2,))` la puerta: cada ejemplo trae 2 números

`Dense(16, activation="relu")` 16 neuronas conectadas con todo; ReLU, la válvula

`Dense(1, activation="sigmoid")` la salida: una probabilidad

`model.summary()` cuenta las perillas: **48 + 272 + 17 = 337**.



Entrada Oculta 1 Oculta 2 Salida

Compilar: las reglas del entrenamiento

`compile` todavía no entrena. Decide cómo se corrige, qué se mide y qué te reporta, como planear el estudio antes de empezar.

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
```

optimizer · cómo da los pasos

Adam, el de la carrera, con `learning_rate` = η = 0.01: el largo del paso del viernes.

loss · qué tan mal va

la BCE del viernes: castiga la falsa seguridad, como al meteorólogo.

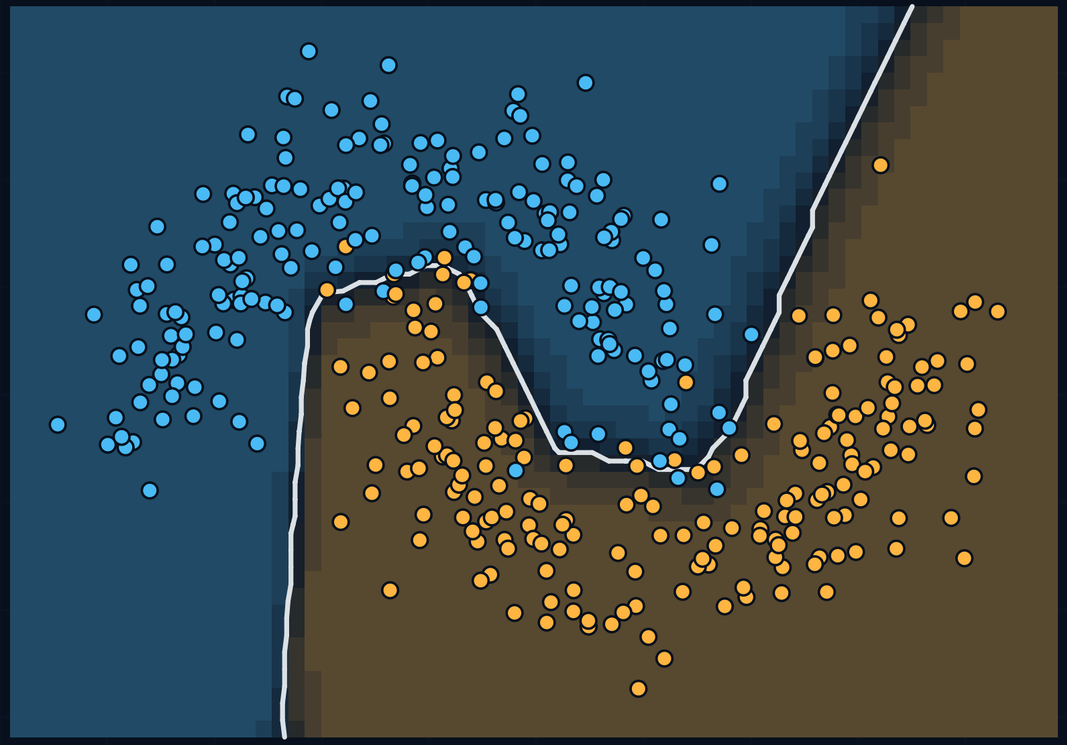
metrics · qué te reporta

la exactitud. Solo informa, no se usa para aprender. Ojo con el 87 % del bloque 09.

Entrenar con `fit`

```
history = model.fit(X_train, y_train, epochs=60, batch_size=32, validation_split=0.2)
```

`fit` repite adelante, atrás y actualizar. `epochs=60` 60 vueltas · `batch_size=32` 12 pasos por vuelta · `validation_split=0.2` el simulacro



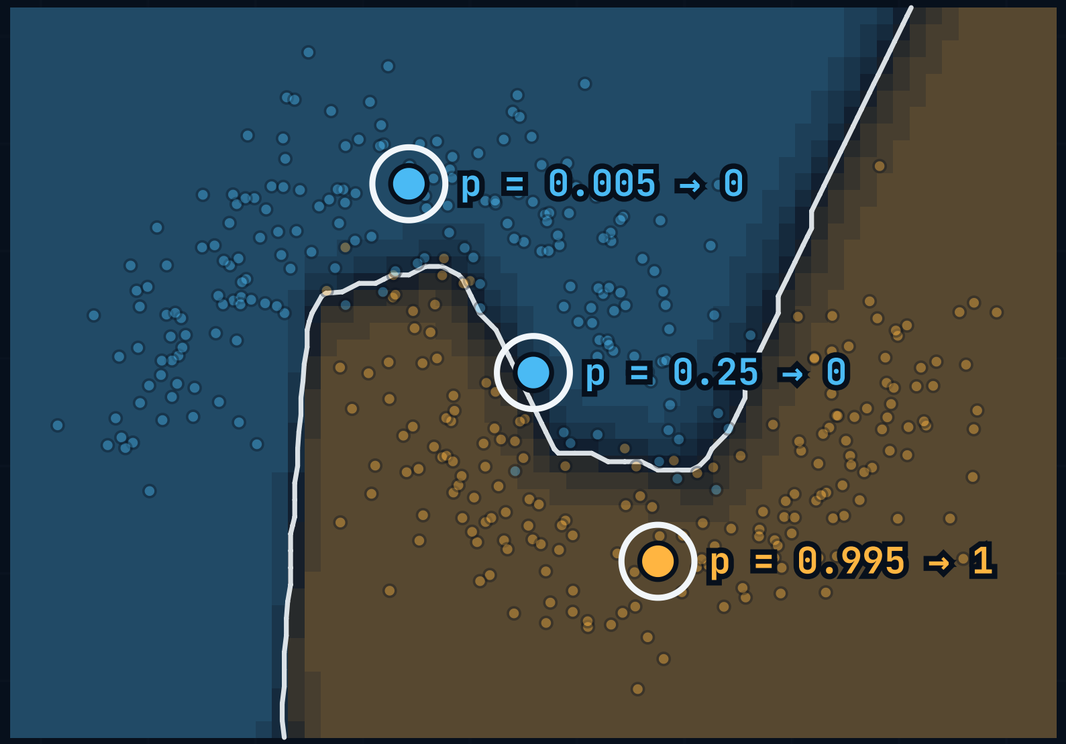
época **60**/60 paso **720**/720 pérdida **0.09**
simulacro **96** %

Entrenar 60 épocas **Reiniciar**



Predecir: de probabilidad a decisión

`predict` da una probabilidad y el umbral 0.5 la convierte en sí o no. `evaluate` toma el examen final.




```
p = model.predict(NUEVOS)        # 0..1
clase = (p ⩾ 0.5).astype(int)    # 0 / 1
model.evaluate(X_test, y_test)   # exam
```

```
salida  p = [0.005 0.995 0.252]
        clase = [0 1 0]
```

Examen final: 120 ejemplos que la red nunca vio.
Exactitud **95 %**.

El umbral 0.5 no es sagrado: en el bloque 09 lo movemos.

Todo junto: tu primera red en Keras

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")])
model.compile(optimizer=keras.optimizers.Adam(0.01),
              loss="binary_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=60,
                    batch_size=32, validation_split=0.2)
p = model.predict(NUEVOS)
```

1 · Construir apilar capas Dense: ReLU dentro, sigmoide a la salida

2 · Compilar Adam + la BCE del viernes + exactitud

3 · Entrenar fit = adelante, atrás y actualizar, por lotes y épocas

4 · Predecir probabilidad → umbral → decisión

Diez líneas, **337** parámetros y **95 %** en el examen final.

Práctica en clase: NB02

En clase · sin nota

40 minutos

En parejas. Cada parte explica el código línea por línea; las partes A a D repiten con código las láminas, por eso los números coinciden.

Qué deberías ver

- **T** · Importa TensorFlow y Keras; tu primer tensor
- A · Forward: **0.56** y pérdida **0.59** · B · culpas **–0.44, –0.27, –0.28**
- C · Un paso: **56 %** → **65 %** · D · TensorFlow coincide con tu cuenta
- E · Keras paso a paso: **XOR resuelto** con 33 parámetros
- F · Las lunas: **337** parámetros y **95 %** en el examen final
- **Mini-reto 2** · los círculos con más de **90 %**

deeplearningweek1.vercel.app/?nb02

Archivo → **Guardar una copia en Drive** y ejecuta primero **Configuración**

09

¿El modelo es bueno?

Cuando la exactitud engaña y qué mirar en su lugar.

87 % que no vale nada

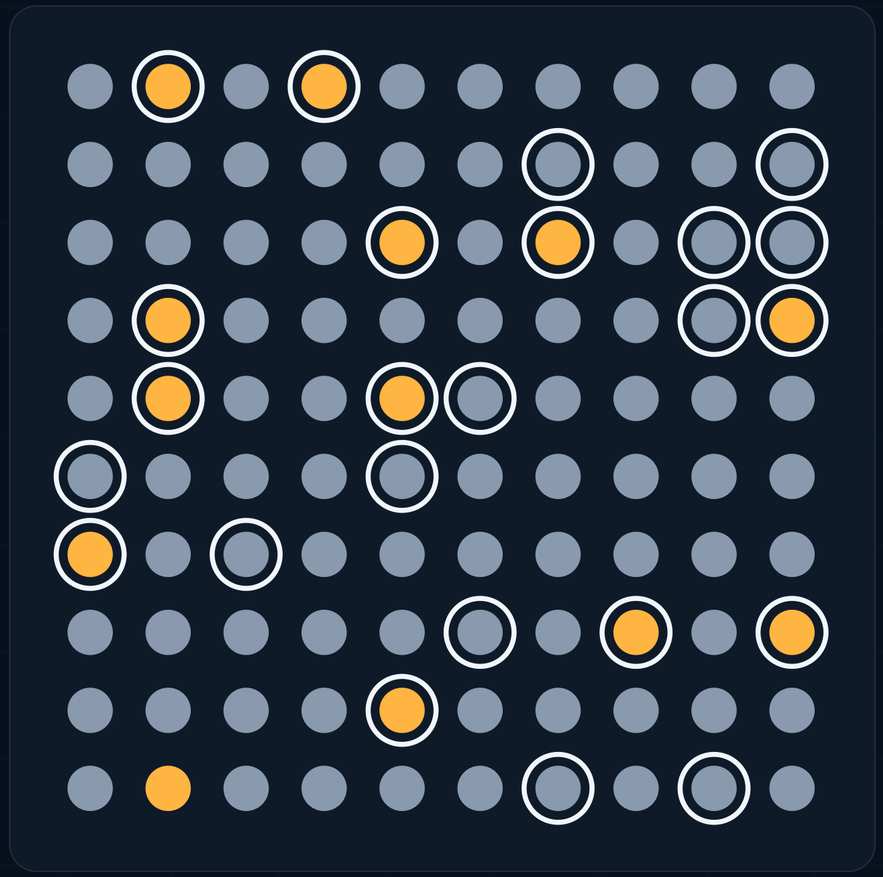
Recall, precisión y AUC

Sábado · 15 minutos

87 % de exactitud puede no valer nada

Idea clave 5

Una cooperativa busca a los socios que caerán en mora: 13 de cada 100. Un modelo que nunca avisa es como una alarma de humo que nunca suena: casi siempre «acierta».



Una red de verdad: marca a quien supera el umbral

exactitud	87 %
morosos encontrados	12 de 13
falsas alarmas	12

umbral 0.50

cae en mora · al día · el modelo lo marca

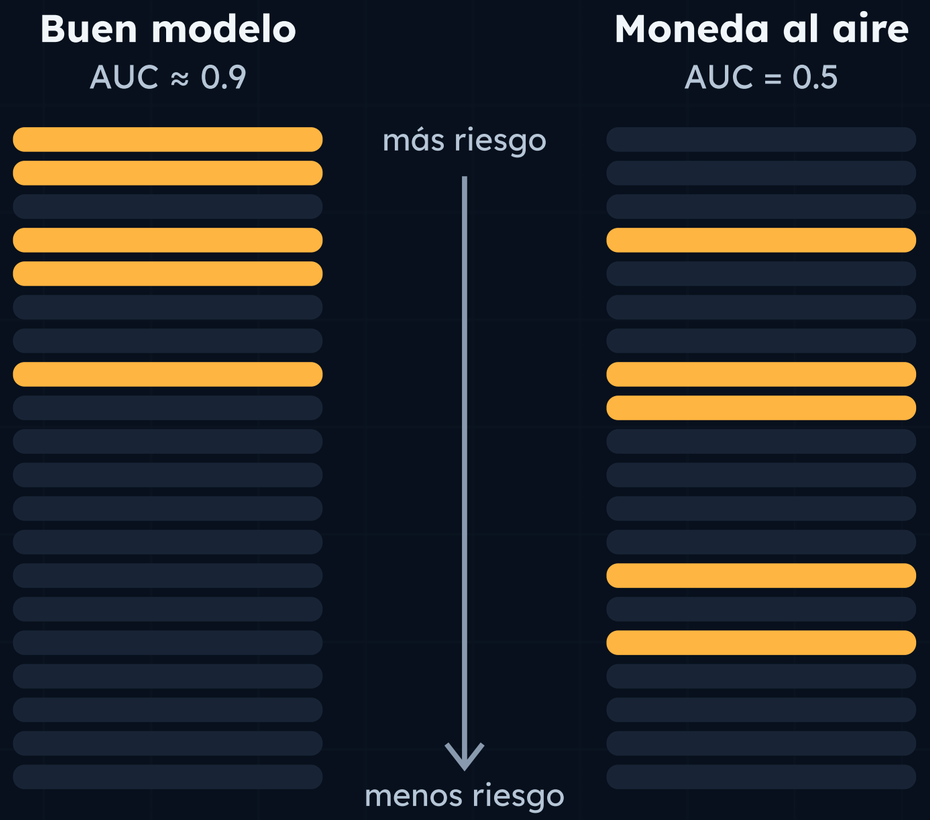
Mira más allá de la exactitud

Idea clave 5

Tres preguntas mejores cuando los casos que buscas son pocos.

- **R** **Recall:** de los que caen en mora, ¿cuántos encontramos?
- **P** **Precisión:** de los que marcamos, ¿cuántos sí caen?
- **A** **AUC:** ¿qué tan bien ordena el riesgo? 0.5 es tirar una moneda; 1 es perfecto.

Años 40 la curva ROC y el AUC nacen con el radar en la Segunda Guerra Mundial



Quiz final: ¿cuánto avanzamos? No es nota

10 preguntas tipo Kahoot. Vuelven las 5 ideas clave del pre-quiz: así vemos tu avance.

1. Abre **deeplearningweek1.vercel.app/?final**
2. Usa **el mismo nombre** que en el pre-quiz
3. Pega tu código en el chat de Teams

deeplearningweek1.vercel.app/?final

De cero a tu primera red

Todo lo de la semana, en el orden en que lo vimos.



Borde amarillo: las 5 ideas clave

Todo lo que vimos tiene fecha

Casi todas las ideas de la semana tienen décadas, algunas siglos. Lo que cambió es cuánto cómputo y cuántos datos tenemos.

1676 — Leibniz · la regla de la cadena

1805 — Legendre · el MSE

1847 — Cauchy · bajar por la pendiente

1943 — McCulloch y Pitts · la neurona

1958 — Rosenblatt · el perceptrón

1969 — Minsky y Papert · XOR

1986 — Rumelhart y Hinton · retropropagación

2010 — ReLU y buenos pesos iniciales

2014 — Kingma y Ba · Adam

2015 — TensorFlow y Keras

2024 — Muon · llega a Kimi K2

2026 — Tú · tu primera red

Si las ideas son viejas, ¿qué hace avanzar al deep learning: **mejores algoritmos o más cómputo**? Es la pregunta de la lectura de la semana.

Lo que viene: semanas 2 y 3 Lecturas de tendencia

Las próximas lecturas de tendencia salen de preguntas que hoy ya puedes hacerte.

Semana 2

¿El regreso de las redes recurrentes? Transformers, Mamba y la búsqueda de la arquitectura ideal

Hoy la red miró todos los datos a la vez. ¿Y si llegan en orden, como las palabras de una frase o las ventas de cada día?

Semana 3

¿Puedes explicar por qué tu modelo dijo «no»?

El socio B del NB01 recibió un «no» (z = −3.61). En Ecuador, desde marzo de 2026, una norma de la Superintendencia de Protección de Datos permite pedir una explicación de la lógica del algoritmo.

Para repasar y profundizar

- **Guía de estudio de la semana:** los 9 temas con ejemplos y ejercicios
- **Nielsen (2015)**, *Neural networks and deep learning*, capítulos 1 y 2 · neuralnetworksanddeeplearning.com
- **Zhang, Lipton, Li y Smola (2023)**, *Dive into deep learning*, «Multilayer perceptrons» · d2l.ai
- **TensorFlow Playground**: entrena redes en el navegador · playground.tensorflow.org
- Los notebooks NB01 y NB02 quedan en la página para practicar cuando quieras

deeplearningweek1.vercel.app/?guia

Gracias

En el chat, del 1 al 5: **¿podrías explicarle a un colega cómo aprende una red?**

Andrés Ordóñez-Bolaños · Neural Networks: Deep Learning · UIDE